A Scalable Coercion-Resistant Voting Scheme for Blockchain Decision-Making

Zeyuan Yin, Bingsheng Zhang, Andrii Nastenko, Roman Oliynykov, Kui Ren

***Abstract*—Typically, a decentralized collaborative blockchain decision-making mechanism is realized by remote voting. To date, a number of blockchain voting schemes have been proposed; however, to the best of our knowledge, none of these schemes achieve coercion-resistance. In particular, for most blockchain voting schemes, the randomness used by the voting client can be viewed as a witness/proof of the actual vote, which enables improper behaviors such as coercion and vote-buying. Unfortunately, the existing coercion-resistant voting schemes cannot be directly adopted in the blockchain context. In this work, we design the first scalable coercion-resistant blockchain voting scheme that supports private weighted votes and 1-layer liquid democracy as introduced by Zhang *et al.* (NDSS '19). Its overall complexity is $O(n)$, where n is the number of voters. Moreover, the ballot size is reduced from Zhang *et al.*'s $\Theta(m)$ to $\Theta(1)$, where m is the number of experts and/or candidates. We formally prove that our scheme has ballot privacy, verifiability, and coercion-resistance. We implement a prototype of the scheme, and the evaluation results show that our scheme's tally procedure is more than 6x faster than VoteAgain (USENIX '20) in an election with over 50,000 voters and over 50% extra ballot rate.**

***Index Terms*—Electronic voting, coercion-resistant, blockchain decision-making, remote voting, liquid democracy**

I. Introduction

Blockchain technology has enjoyed its popularity since the invention of Bitcoin in 2008, and it continues to reshape our digital society with its properties of decentralization, transparency, and security. Democracy on the blockchain has the potential to transform the way political systems function, creating a more equitable and inclusive future.

Democracy on the blockchain. In a democratic blockchain system, stakeholders have the right to participate in the decision-making process through verifiable remote voting, where everyone can express his opinion. Usually, in blockchain voting, each participant's voting power is different and proportional to his stake, which is called *weighted votes*. This is one of the main differences between blockchain voting and traditional elections, where, typically, one participant has one vote. On the other hand, direct democracy might not always be the best choice for a blockchain decision-making process. In practice, to make a wise decision, stakeholders must invest substantial effort into acquiring knowledge and expertise throughout the decision-making process. Therefore, letting elites lead the decision-making might be an optimization in most cases. Some systems, such as ZCash [1], use a small committee (consisting of several experts) to make the decisions; however, this has the risk of centralization, i.e., if the committee behaves maliciously, there is no mechanism for stakeholders to alter their decisions whatsoever.

The concept of liquid democracy has been proposed to achieve better collaborative intelligence. Liquid democracy (also known as delegative democracy [2]) is a hybrid of direct democracy and representative democracy. It provides the benefits of both systems (whilst avoiding their drawbacks) by enabling organizations to take advantage of experts in a blockchain voting process, as well as giving the stakeholders the opportunity to vote. For each proposal, a voter can either vote directly or delegate his voting power to an expert who is knowledgeable and renowned in the corresponding area.

Zhang *et al.* [3] proposed a treasury system that supports liquid democracy. However, their scheme has the following two drawbacks: (i) the ballot size is linear in the number of candidates and/or experts; (ii) it is not coercion-resistant.

The coercion problem in remote voting. In real-world voting, a voting booth gives a voter privacy and protects him from being coerced. However, in remote voting, the voting procedure can be viewed as a probabilistic algorithm that takes as input a random coin and the voter's choice. If the output is published on the bulletin board, then we have a problem: the input and randomness used in the voting procedure can be viewed as proof of casting a certain ballot, and anyone can rerun the probabilistic algorithm to verify it, which makes coercion and vote-buying possible. Many well-known e-voting systems are not coercion-resistant, such as snapshot [4], Helios [5], and prêt à voter [6]. What's worse, in the blockchain context, vote-buying becomes easier with the help of smart contracts.

To address the coercion/vote-buying problem, several schemes are proposed. Generally, coercion-resistant voting can be divided into three categories: fake credentials [7], [8], [9], [10], [11], [12], [13], [14], re-voting [15], [16], [17], [18], and secure hardware [19], [20]. In a coercion-resistant voting scheme using *fake credentials*, a voter has a process for generating fake credentials. If coerced, a voter will cast a ballot using a fake credential, which is indistinguishable from the real one in the coercer's view, and it will be silently and verifiably uncounted in the tally phase. In a *re-voting* scheme, a voter can cast his ballot multiple times, and obliviously only the last one will be tallied; coercion-resistance relies on the fact that the voter can cast the ballot again after the coercer leaves. In the schemes using *secure hardware*, the secure hardware has its internal randomness source; it can do probabilistic encryption

Zeyuan Yin, Bingsheng Zhang, and Kui Ren are with The State Key Laboratory of Blockchain and Data Security, Zhejiang University, and with Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security. Emails:{zeyuanyin,bingsheng,kuiren}@zju.edu.cn

Andrii Nastenko is with IOG Singapore Pte Ltd, and with Kharkiv National University of Radio Electronics, Ukraine. Email: andrii.nastenko@iohk.io

Roman Oliynykov is with IOG Singapore Pte Ltd, and with V.N.Karazin Kharkiv National University, Ukraine. Email: roman.oliynykov@iohk.io

for the voter so that the voter can lie about what has been encrypted.

Coercion-resistance v.s. deniability. All types of coercion-resistant voting schemes must give a voter deniability in a certain way. Concretely, in *fake credentials* schemes, the election authority will provide randomness in the credential-related elements and generate a designated verifier proof of correctness. To deceive the coercer, a voter can generate a fake credential and claim it as the real one by simulating the designated verifier proof. Namely, the registration procedure is deniable. In *re-voting* schemes, the re-vote operation must be deniable, i.e., the tally procedure will not reveal if a voter has re-voted. In the schemes using *secure hardware*, the secure hardware hides the randomness in the ciphertext so that a voter can claim that it is the encryption of another candidate, i.e., the encryption operation is deniable.

Challenges. Could we apply the aforementioned techniques to realize a coercion-resistant voting scheme in the blockchain context? It turns out to be a non-trivial task. First, secure hardware-based solutions might not be suitable for the blockchain setting because an open blockchain allows anyone to join and leave freely, and not all devices are equipped with secure hardware, such as a trusted execution environment (TEE).

How about "fake credentials" and "re-voting" schemes? Can we adapt those schemes with weighted votes? There are still some challenges. For instance, JCJ [7] is a well-known coercion-resistance voting scheme, and it can be modified to support weighted votes; however, the scheme has $O(n^2)$ complexity due to the pair-wise plaintext equivalence tests (PETs), where n is the total number of votes, limiting its scalability. On the other hand, although the recently proposed scheme, VoteAgain [18], offers quasi-linear complexity, its verifiability relies on a trusted third party (TTP), which is undesirable in the blockchain setting. Araújo *et al.*'s "fake credentials" scheme [14] achieves $O(n)$ complexity without relying on TTP for verifiability. Unfortunately, the credentials of Araújo *et al.*'s scheme are in the form of two group elements satisfying a linear relationship, and this makes it unable to be modified trivially to support weighted votes. To the best of our knowledge, no proper coercion-resistant voting scheme in the literature can support weighted votes and achieve $O(n)$ complexity at the same time. Hereby, we are asking the question:

> *Can we design a scalable (linear complexity) coercion-resistant delegated voting scheme for blockchain decision-making?*

A. Our Approach

In this work, we answer the above question affirmatively by proposing a new coercion-resistant voting scheme. Our scheme belongs to the "fake credentials" category. We start with the well-known JCJ scheme [7]. In the JCJ scheme, each valid (encrypted) credential is put on the bulletin board in the registration phase, and each ballot generally consists of an encrypted candidate and an encrypted credential. In the tally phase, by a shuffle and pair-wise PETs on the credentials, the ballots with fake credentials will be silently eliminated.

Intuitively, one can associate credentials with voting power in the JCJ scheme to support weighted votes, i.e., each encrypted credential is tied with encrypted voting power, and we still perform PETs on the encrypted credentials in the tally phase. However, the scheme will have $O(n^2)$ complexity, so it does not scale well when the number of voters is large. To improve scalability, we propose a novel *"dummy voting power"* technique. The key idea is that we allow voters to publish (encrypted) fake credentials associated with (encrypted) zero voting power on the bulletin board. Then, in the tally phase, after shuffle re-encrypting the real and fake credentials, all the credentials can be decrypted. In this way, we transform the pair-wise PETs into "decrypt and match", achieving $O(n)$ complexity.

To achieve delegation, we design a *"two-layer homomorphic tally"* procedure consisting of "delegation calculation" and "final tally calculation". In layer one, delegation is calculated by decrypting voters' choices and adding the delegated voting power to the corresponding experts. In layer two, the final tally result is calculated by decrypting experts' choices and adding experts' voting power together with voters' direct votes. Thanks to the additive homomorphism of the encryption scheme, voters' ballots and voting power are hidden throughout the tally.

Combining the "dummy voting power" technique and "two-layer tally" procedure, we build the first coercion-resistant voting scheme that has linear complexity and supports private weighted votes and liquid democracy. We formally prove that our scheme meets the game-based definitions of ballot privacy, verifiability, and coercion-resistance. We implement the scheme and evaluate its performance. Results show that our scheme's tally procedure is more than 3.45x faster than VoteAgain [18] in elections with over 50,000 voters and over 50% extra ballot rate[1].

B. Related Work

Coercion-resistant voting can be roughly split into three classes: fake credentials [7], [8], [9], [10], [11], [12], [13], [14], re-voting [15], [16], [17], [18], and secure hardware [19], [20]. We briefly survey them below.

JCJ [7] is the first paper that introduces the *"fake credentials"* type of coercion-resistant voting, and it was implemented as Civitas [9]. In JCJ, each ballot contains an encrypted credential and the encrypted choice, and there is a list of encrypted valid credentials on the bulletin board. In the tally phase, by pair-wise PETs on the encrypted credentials, the ballots with invalid credentials will be eliminated, but the pair-wise PETs cause $O(n^2)$ complexity, where n is the number of voters. To improve JCJ's complexity, Araújo *et al.* proposed a series of work [8], [13], [14] that employs credentials with special structure to achieve linear complexity. Another approach to achieving linear work is based on generating a set of fake credentials (or panic passwords) in advance [10], [12].

[1]In VoteAgain, it means that more than 50% voters re-voted once; in our scheme, it means that more than 50% voters cast a fake ballot.

TABLE I: Comparison of voting schemes. Here, n is the number of voters, and m is the number of election candidates. Del. means delegation. Crypto state means that the voter needs to keep a cryptographic secret from the coercer. EA stands for election authority. • means it does not achieve full coercion-resistance. Hardware means that the property is guaranteed by secure hardware. An item in bold text means that our scheme is the best in this aspect.

Schemes	Weighted votes	Del.	Ballot size	Complexity	Crypto state	Ballot privacy	Verifiability	Coercion-resistance
JCJ [7] (fake credentials)	No	No	$O(1)$	$O(n^2)$	Yes	t-out-of-k	trust no one	trust EA, Yes
KHF [10] (fake credentials)	No	No	$O(1)$	$O(n)$	Yes	t-out-of-k	trust no one	trust EA, •
ABBT [14] (fake credentials)	No	No	$O(1)$	$O(n)$	Yes	t-out-of-k	trust no one	trust EA, Yes
AKL+ [16], LHK [17] (re-voting)	No	No	$O(1)$	$O(n^2)$	No	t-out-of-k	trust no one	secret credential, Yes
VoteAgain [18] (re-voting)	No	No	$O(1)$	$O(n \log n)$	No	t-out-of-k	trust EA	trust EA, Yes
MBC [19], AOZZ [20] (secure hardware)	No	No	$O(1)$	$O(n)$	No	hardware	hardware	hardware, Yes
Snapshot [4]	Yes	No	$O(1)$	$O(n)$	-	t-out-of-k	trust no one	No
ZOB [3]	Yes	Yes	$O(m)$	$O(mn)$	-	t-out-of-k	trust no one	No
Our scheme (fake credentials)	**Yes**	**Yes**	$\boldsymbol{O(1)}$	$\boldsymbol{O(n)}$	Yes	$\boldsymbol{t}$**-out-of-**$\boldsymbol{k}$	**trust no one**	trust EA, Yes

However, this approach will weaken the coercion-resistance level because the number of fake credentials is fixed. Specifically, if every voter has d fake credentials, the coercer can force the voter to cast $d+1$ ballots for the same candidate to break coercion-resistance. Even if the number of fake credentials has a special distribution, it still cannot achieve *full* coercion-resistance as defined in the literature, i.e., they cannot prove that the coercer's advantage of catching a deceiving voter is negligible. Spycher *et al.* [11] modifies the JCJ scheme by requiring the voter to indicate the corresponding credential in the ballot to achieve $O(n)$ complexity; however, the scheme does not achieve *full* coercion-resistance, either.

The *re-voting* type of coercion-resistant voting allows a voter to cast multiple ballots, and the tally procedure will obliviously only count the last one. Achenbach *et al.* [16] and Locher *et al.* [17] utilize a deniable vote update mechanism to realize re-voting with quadratic complexity. The Norwegian Internet voting protocol [15] and VoteAgain [18] achieve (quasi-)linear complexity, but they both need a trusted third party for verifiability. Schemes based on *secure hardware* [19], [20] can achieve coercion-resistance easily, but secure hardware is a strong assumption and is usually not suitable in practice. A recent work by Giustolisi *et al.* [21] proposes the first re-voting scheme that can evade last-minute voter coercion. The scheme has a re-voting form, but it requires the voter to remember a secret (number of cast ballots), so it has similar assumptions as "fake credentials" schemes. The scheme can be modified to support weighted votes; however, in the scheme, the voting server needs to continuously generate obfuscation ballots for each voter, so its complexity is linear to the duration of the voting phase and does not achieve $O(n)$ complexity.

Recently, Cortier *et al.* [22] observe a problem with the JCJ voting scheme that it leaks the re-voting pattern when re-voting is allowed, and they propose a variant of JCJ that can hide the re-voting pattern. Our scheme does not allow re-voting, so it is not affected by this problem.

On the other hand, blockchain voting [23], [4], [3], [24] is becoming more and more popular nowadays. Snapshot [4] is a popular DAO (Decentralized Autonomous Organization) voting platform that frees voters from gas fees. It uses IPFS [25] to store the proposals and votes, making the voting process off-chain and gas-free. Zhang *et al.* [3] propose a treasury system for blockchain governance. It supports liquid democracy and is provably secure, but its ballot size is linear to the number of candidates. Besides, to the best of our knowledge, none of the existing blockchain voting schemes are coercion-resistant.

Finally, Table I gives a comparison between our scheme and previous work.

II. System Overview

In this section, we start by modifying the JCJ protocol [7] to build a coercion-resistant voting scheme that supports weighted votes with $O(n^2)$ complexity. Then, we provide the intuition and details of our novel "dummy voting power" technique and "two-layer tally" procedure. We also optimize JCJ's ballot structure to achieve a smaller ballot size. Finally, we discuss the blockchain deployment of our scheme.

A. The JCJ Protocol

We first recall the well-known JCJ protocol [7] and show how to modify it to support weighted votes.

The original protocol. As mentioned above, JCJ is the first protocol that introduces the concept of "fake credentials". Generally, it works as follows. For simplicity, we use $[\![x]\!]$ to denote an encryption of x in section II-A and II-B. In the registration phase, a voter authenticates to the election authority (EA), and the EA generates a credential $\sigma \leftarrow \mathbb{G}$. Then, the EA publishes $S = [\![\sigma]\!]$ on the BB (bulletin board) and sends σ to the voter along with a designated verifier proof that S is an encryption of σ. In the voting phase, a voter casts a ballot $B = ([\![v]\!], [\![\sigma]\!], Pf)$ where $[\![v]\!]$ is the encryption of a candidate v, $[\![\sigma]\!]$ is the encryption of a credential σ, Pf includes the NIZK proofs of knowledge of v and σ, and a NIZK proof that $[\![v]\!]$ encrypts a valid candidate. If a voter is coerced, he generates a random $\sigma' \leftarrow \mathbb{G}$ and claims it as the real credential by simulating the designated verifier proof. In the tally phase, the trustees shuffle the ballots and perform pair-wise PETs on the encrypted credentials to eliminate the ballots with fake credentials. Finally, the trustees decrypt the candidates and tally the votes. The overview of the JCJ protocol is shown in Fig. 1.

Supporting weighted votes. We can see that if we associate each credential with voting weight, then the system can easily

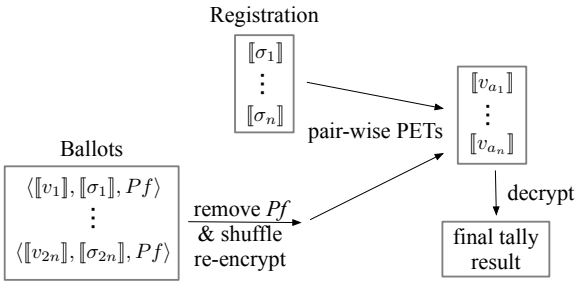

Fig. 1: JCJ original protocol, where $[\![\cdot]\!]$ denotes encryption, σ_i represents a credential, v_i represents a candidate, and Pf represents the ZKP.

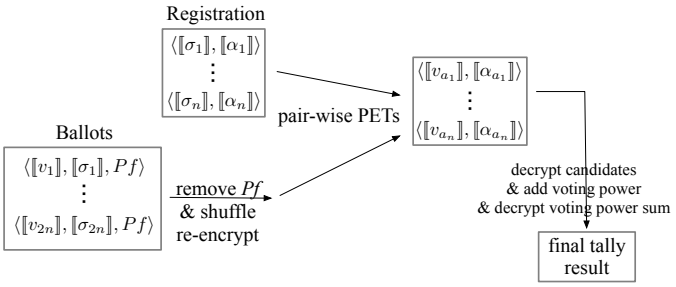

Fig. 2: JCJ protocol with weighted votes, where $[\![\cdot]\!]$ denotes encryption, σ_i represents a credential, v_i represents a candidate, α_i represents voting power, and Pf represents the ZKP.

support weighted votes. More specifically, in the registration phase, the EA publishes $S = ([\![\sigma]\!], [\![\alpha]\!])$ on the BB, where $[\![\sigma]\!]$ is the encrypted credential and $[\![\alpha]\!]$ is the encrypted voting power under an additively homomorphic encryption scheme. In the tally phase, the trustees use pair-wise PETs to match the (encrypted) candidate and (encrypted) voting power, and get tuples of $\langle[\![v]\!], [\![\alpha]\!]\rangle$, where $[\![v]\!]$ is the encrypted candidate and $[\![\alpha]\!]$ is the encrypted voting power. Then, the trustees decrypt the candidates and add the voting power by additive homomorphism. The overview of the modified JCJ protocol with weighted votes is shown in Fig. 2.

B. Our Technique

In this part, we will illustrate our novel techniques for achieving $O(n)$ complexity and delegated voting.

Dummy voting power. We can see that in the JCJ protocol, the pair-wise PETs cause $O(n^2)$ complexity. The idea is that if we allow voters to publish the fake credentials on the BB but associate them with dummy (zero) voting power, then we can directly decrypt the credentials instead of performing PETs in the tally phase. In other words, in the registration phase, the EA publishes (encrypted) real credentials and (encrypted) real voting power; at any convenient time before the tally phase, voters can also publish (encrypted) fake credentials and (encrypted) dummy voting power. In this way, we switch pair-wise PETs into shuffle-decrypt and matching, achieving linear complexity. Furthermore, "dummy voting power" also hides the number of votes obtained by each candidate. The idea of "dummy voting power" technique is shown in Fig. 3.

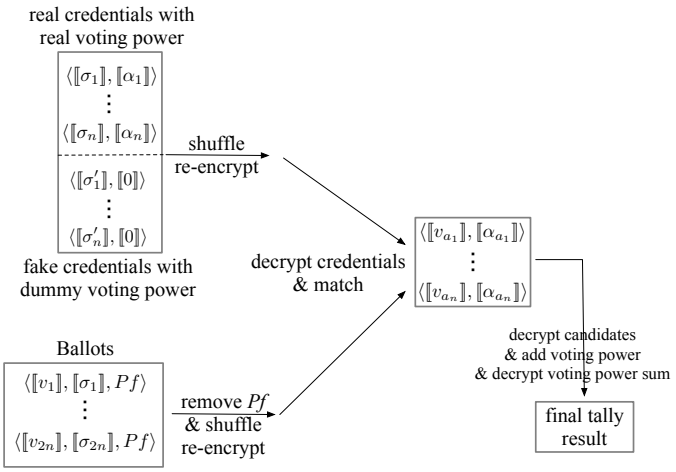

Fig. 3: Dummy voting power technique, where $[\![\cdot]\!]$ denotes encryption, σ_i represents a credential, v_i represents a candidate, α_i represents voting power, and Pf represents the ZKP.

Two-layer tally. To support delegation, the trustees perform a "two-layer tally" procedure in the tally phase. Generally speaking, in layer one, voters' choices are decrypted, and the delegated voting power is added to the corresponding experts. In layer two, experts' choices are decrypted, and the final tally result is calculated by adding experts' voting power and voters' direct votes. Note that experts have input independence instead of ballot privacy, so their ballots can be decrypted directly. Fig. 4 shows the process of "two-layer tally" where there are 3 candidates and 2 experts.

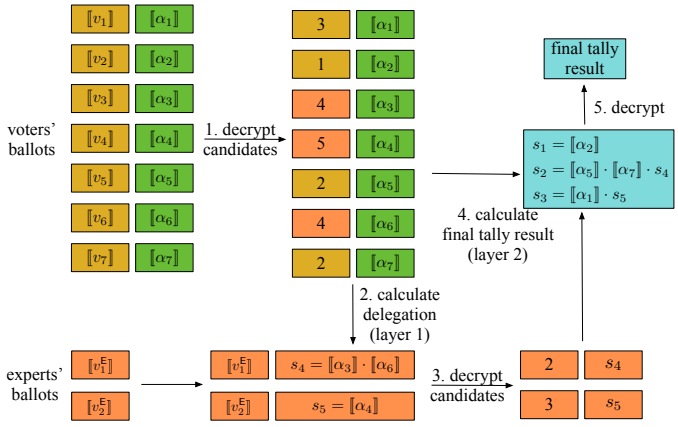

Fig. 4: Two-layer tally, where $[\![\cdot]\!]$ denotes encryption, α_i represents voting power, s_4, s_5 represent the (encrypted) delegated voting power, and s_1, s_2, s_3 represent the (encrypted) final result.

An optimization of JCJ's ballot structure. We further optimize JCJ's ballot structure in the following two aspects. Firstly, we observe that it is not necessary to prove that $[\![v]\!]$ encrypts a valid candidate because all of them will be decrypted in the tally phase. If it encrypts an invalid value, we can simply treat it as "abstain" and drop it. Secondly, instead of defining σ as the secret credential, we can define the

discrete logarithm of σ as the secret credential (voting secret key) and define σ as the voting public key, i.e., $\sigma := \mathsf{pk} := g^{\mathsf{sk}}$. Then, the ballot can be modified as $\langle \mathsf{pk}, \boldsymbol{u}, \pi, Sig \rangle$, where $\boldsymbol{u} := [\![v]\!]$ is the encrypted choice, π is a NIZK proof of plaintext knowledge of $\boldsymbol{u}$, and $Sig \leftarrow \mathsf{Sign}(\mathsf{sk}, \boldsymbol{u})$. By doing so, we change an Elgamal ciphertext $[\![\sigma]\!]$ to a group element pk, achieving a smaller ballot size.

C. Blockchain Deployment

Now, we discuss the deployment of our scheme on the blockchain.

Roles. There are five roles in the protocol: voters, experts, registration authority (RA), shuffler, and trustees.

- *A voter* has a certain amount of voting power and can either vote on the proposal directly or delegate his voting power to an expert.
- *An expert* does not have voting power himself, but he can be delegated to vote on others' behalf.
- *The RA* is responsible for the registration procedure.
- *The shuffler* performs verifiable shuffle procedures on the ciphertexts.
- *The trustees* are responsible for decrypting the ballot and revealing the final tally result.

To deploy the scheme on a blockchain, the key point is how the authentication works. Generally speaking, in the registration phase, a voter will first freeze his stake by a transaction tx. Then, he sends a registration request $\langle K, A, \mathsf{tx}, \delta \rangle$ to the RA, where $K \leftarrow \mathsf{EC.Enc}_{\mathsf{pk}_\mathsf{T}}(\mathsf{pk})$ is the encryption of real public key, $A \leftarrow \mathsf{LE.Enc}_{\mathsf{pk}_\mathsf{T}}(\alpha)$ is the encryption of voting power, and δ is the NIZK proof that the encrypted voting power equals the frozen stake. By the NIZK proof δ, we can ensure that the voter's voting power is correct without revealing it. We will show the details in section III.

In the following, we discuss how the RA, shuffler, and trustees are selected and who plays the role of a validator in checking the election.

- *The RA.* The RA is trusted for coercion-resistance and cannot be distributed (see sec. IV for details). Therefore, it may be instantiated with a trusted execution environment (TEE) like Intel SGX.
- *The shuffler.* The shuffler can be implemented by a mixnet [26], and the mixnet nodes can be selected by cryptographic sortition [27]. Ballot privacy is preserved as long as one mixnet node is honest.
- *The trustees.* The trustees can also be selected by cryptographic sortition [27]. In the blockchain context, when the majority of the stake is honest, the majority of trustees are honest with a high probability.
- *The validator.* Every participant can be the validator to check all the NIZK proofs if he wants. Since our scheme includes shuffle proofs, whose verification cost is relatively heavy, it is not recommended to deploy a smart contract to play the role of the validator.
- *Roles of the blockchain.* In our scheme, the blockchain serves as the BB. It also plays the role of PKI in the registration phase to authenticate voters and experts.

III. The Protocol

In this section, we will give a detailed protocol description of our voting scheme.

A. Cryptographic primitives

Notations. Let $\lambda \in \mathbb{N}$ be the security parameter. Let $\mathbb{G}$ be a cyclic group of prime order p with group generator g. We denote the integers modulo p with $\mathbb{Z}_p$ and write $r \leftarrow_\$ \mathbb{Z}_p$ for r being chosen uniformly from $\mathbb{Z}_p$. We abbreviate *probabilistic polynomial time* as PPT.

(Lifted) ElGamal Encryption. ElGamal encryption scheme [28] consists of three PPT algorithms: the key generation algorithm $\mathsf{EC.Keygen}(\mathbb{G}, g, p)$ takes as input the group parameters and outputs a public-private key pair $(\mathsf{pk} := g^{\mathsf{sk}}, \mathsf{sk})$; the encryption algorithm $\mathsf{EC.Enc}_{\mathsf{pk}}(m; r)$ takes as input the public key pk, the message $m \in \mathbb{G}$, and randomness $r \leftarrow_\$ \mathbb{Z}_p$, and it outputs the ciphertext $c := (c_1, c_2) := (g^r, m \cdot \mathsf{pk}^r)$; the decryption algorithm $\mathsf{EC.Dec}_{\mathsf{sk}}(c)$ takes as input the secret key sk and the ciphertext c and outputs the message $m := c_2 / c_1^{\mathsf{sk}}$.

ElGamal encryption is a re-randomizable encryption scheme. The re-encryption algorithm $\mathsf{EC.Rand}_{\mathsf{pk}}(c; r)$ takes as input a ciphertext $c := (c_1, c_2)$ and randomness $r \leftarrow_\$ \mathbb{Z}_p$, and it outputs the re-randomized ciphertext $c' := (c_1', c_2') := (g^r \cdot c_1, \mathsf{pk}^r \cdot c_2)$.

Lifted ElGamal encryption is a variant of ElGamal encryption. The encryption algorithm $\mathsf{LE.Enc}_{\mathsf{pk}}(m; r)$ takes as input the public key pk, the message m, and randomness $r \leftarrow_\$ \mathbb{Z}_p$, and it outputs the ciphertext $c := (c_1, c_2) := (g^r, g^m \cdot \mathsf{pk}^r)$; the decryption algorithm $\mathsf{LE.Dec}_{\mathsf{sk}}(c)$ takes as input the secret key sk and the ciphertext c and outputs the message $m := \mathsf{Dlog}(c_2 / c_1^{\mathsf{sk}})$, where $\mathsf{Dlog}(x)$ outputs the discrete logarithm of x (note that computing the discrete logarithm is inefficient, thus the message space should be small in practice).

The (lifted) ElGamal encryption scheme is IND-CPA secure under the DDH assumption (see Appendix A for the formal definition of IND-CPA security). We omit the randomness r when it is not crucial to the context. Lifted ElGamal encryption is additively homomorphic, i.e., $\mathsf{LE.Enc}_{\mathsf{pk}}(m_1) \cdot \mathsf{LE.Enc}_{\mathsf{pk}}(m_2) = \mathsf{LE.Enc}_{\mathsf{pk}}(m_1 + m_2)$. Besides, with distributed key generation [29], (lifted) ElGamal encryption can be used as a threshold encryption scheme.

Signature. A signature scheme Sig is defined by three PPT algorithms: A key generation algorithm $\mathsf{Sig.Keygen}(1^\lambda)$ that generates a public-private key pair $(\mathsf{pk}, \mathsf{sk})$; a signing algorithm $\sigma \leftarrow \mathsf{Sig.Sign}_{\mathsf{sk}}(m)$ that generates a signature on message m; and a verification algorithm $\mathsf{Sig.Verify}_{\mathsf{pk}}(\sigma, m)$ that outputs 1 if and only if σ is a valid signature on m. The existential unforgeability under chosen message attack (EUF-CMA) property of a signature scheme is formally defined in Appendix A.

Non-Interactive Zero-Knowledge Proofs/Arguments (NIZK). Our scheme utilizes Sigma protocols with Fiat-Shamir heuristic [30] to construct non-interactive zero-knowledge proofs/arguments of knowledge. There are six NIZK protocols in our scheme for proving: (i) voting power correctness ($\mathsf{NIZK}_{\mathsf{power}}$); (ii) ElGamal encryption plaintext

knowledge ($\mathsf{NIZK}_{\mathsf{knowledge}}$); (iii) re-encryption correctness ($\mathsf{NIZK}_{\mathsf{DVP\text{-}reenc}}$); (iv) knowledge of secret key ($\mathsf{NIZK}_{\mathsf{sk}}$); (v) shuffle correctness ($\mathsf{NIZK}_{\mathsf{shuffle}}$); and (vi) decryption correctness ($\mathsf{NIZK}_{\mathsf{Dec}}$). For simplicity, we sometimes omit the witness used in $\mathsf{NIZK.Prove}()$ when it is clear in context. We give formal definitions of completeness, simulation-sound extractability, and zero-knowledge in Appendix A, and give the constructions of these NIZK protocols in Appendix B.

Distributed Key Generation. In our scheme, the trustees will run a distributed key generation protocol $\mathsf{Vote.DKeyGen}(\mathbb{G}, t, k)$ for threshold key generation, where $\mathbb{G}$ is the group, t is the corruption threshold, and k is the number of trustees. The protocol outputs a public key pk, and each trustee T_i obtains a share of the secret key. We use the protocol by Gennaro *et al.* [29] to realize the threshold distributed key generation.

Finally, the symbols in the protocol description are summarized in TABLE II.

TABLE II: Notation table

Symbol	Description
ℓ, k, m	Number of candidates, trustees, and experts
t	Corruption threshold
$\mathsf{C}_i, \mathsf{V}_i, \mathsf{E}_i$	Candidate, voter, and expert
pk_T	Trustees' public key
$\mathsf{sk}_{\mathsf{T},i}$	Trustee i's secret key
$(\mathsf{pk}_\mathsf{RA}, \mathsf{sk}_\mathsf{RA})$	RA's key pair
tx	Transaction
σ_RA	RA's signature
α	Stake/Voting power
$(\mathsf{pk}, \mathsf{sk})$	Voter's real key pair
$(\mathsf{pk}', \mathsf{sk}')$	Voter's fake key pair
K	Encrypted (real) public key
$\tilde{K}$	Re-encrypted real public key
K'	Encrypted fake public key
A	Encrypted (real) voting power
A_0	Encrypted dummy voting power
δ, ρ, π, τ	NIZK proof
e_i	Expert E_i's authentication message
v	Voter's/expert's choice
$\boldsymbol{u}, \boldsymbol{c}$	Encrypted choice
$\beta_\mathsf{V}, \beta_\mathsf{E}$	Voter's ballot, expert's ballot
$\boldsymbol{s}$	Encrypted delegated voting power/final result

B. Protocol Description

Preparation Phase:

In the preparation phase, the trustees and the RA prepare their cryptographic materials. Formally:

Procedure 1 (Setup). To setup an election with ℓ candidates $\mathcal{C} := \{\mathsf{C}_1, \ldots, \mathsf{C}_\ell\}$, k trustees, and threshold t. The parties proceed as follows:

1) The trustees run $\mathsf{Vote.DKeyGen}(\mathbb{G}, t, k)$ to generate a public encryption key pk_T and each trustee T_i obtains $\mathsf{sk}_{\mathsf{T},i}$, a share of the decryption key. They publish pk_T.
2) The RA runs $(\mathsf{pk}_\mathsf{RA}, \mathsf{sk}_\mathsf{RA}) \leftarrow \mathsf{Sig.Keygen}(1^\lambda)$. It publishes pk_RA.

Registration Phase:

In the registration phase, a voter authenticates to the RA to publish a "registration item". At any convenient time, it can publish "fake registration items" on the BB. Experts also need to register in this phase.

Procedure 2 ($\mathsf{Reg}(\mathsf{Auth})$). A voter takes as input his blockchain inalienable authentication method Auth (usually by proving knowledge of the blockchain secret key) and does the following:

1) Assume that the voter issued a privacy-preserving transaction tx that locks α stake during the election.
2) The voter runs $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Sig.Keygen}(1^\lambda)$.
3) The voter computes $K \leftarrow \mathsf{EC.Enc}_{\mathsf{pk}_\mathsf{T}}(\mathsf{pk})$ and $A \leftarrow \mathsf{LE.Enc}_{\mathsf{pk}_\mathsf{T}}(\alpha)$.
4) The voter generates $\delta \leftarrow \mathsf{NIZK}_{\mathsf{power}}.\mathsf{Prove}(\mathsf{tx}, A)$ proving that A is an encryption of the amount of stake locked in tx.
5) The voter sends $\langle K, A, \mathsf{tx}, \delta\rangle$ to the RA and uses Auth to authenticate to the RA. The RA checks $\mathsf{NIZK.verify}(A, \mathsf{tx}, \delta) \stackrel{?}{=} 1$. If the check passes, the RA computes $\tilde{K} \leftarrow \mathsf{EC.Rand}_{\mathsf{pk}_\mathsf{T}}(K)$ and generates the signature $\sigma_\mathsf{RA} \leftarrow \mathsf{Sig.sign}_{\mathsf{sk}_\mathsf{RA}}(\tilde{K}||A||\mathsf{tx}||\delta)$. Then, the RA publishes the registration item $\langle \tilde{K}, A, \mathsf{tx}, \delta, \sigma_\mathsf{RA}\rangle$ on the BB.
6) The RA generates $\pi_\mathsf{DVP} \leftarrow \mathsf{NIZK}_{\mathsf{DVP\text{-}reenc}}.\mathsf{Prove}(\tilde{K}, K)$ and sends π_DVP to the voter, where π_DVP is a designated verifier proof of re-encryption correctness.
7) The voter checks $\mathsf{NIZK}_{\mathsf{DVP\text{-}reenc}}.\mathsf{Verify}(\tilde{K}, K, \pi_\mathsf{DVP}) \stackrel{?}{=} 1$. If the check does not pass, he raises a complaint.

Procedure 3 ($\mathsf{FakeReg}()$). At any convenient time, a voter can register a fake key with zero voting power.

1) The voter runs $(\mathsf{pk}', \mathsf{sk}') \leftarrow \mathsf{Sig.Keygen}(1^\lambda)$.
2) The voter computes $K' \leftarrow \mathsf{EC.Enc}_{\mathsf{pk}_\mathsf{T}}(\mathsf{pk}')$ and $A_0 = \mathsf{LE.Enc}_{\mathsf{pk}_\mathsf{T}}(0; 0)$.
3) The voter generates a NIZK proof of knowledge of sk': $\rho \leftarrow \mathsf{NIZK}_{\mathsf{sk}}.\mathsf{Prove}(K', \mathsf{sk}')$.
4) The voter publishes the fake registration item $\langle K', A_0, \rho\rangle$ on the BB.

Procedure 4 ($\mathsf{E\_Reg}(\mathsf{Auth})$). In the registration phase, all users who want to be experts also register using the blockchain authentication method Auth.

1) The user i uses Auth to generate an authencation message e_i and publishes $\langle \mathsf{E}_i, e_i\rangle$ on the BB, where E_i is the expert's identity.

Voting/Delegation Phase:

Procedure 5 ($\mathsf{V\_Vote}(\mathsf{pk}, \mathsf{sk}, v)$). To cast a vote, a voter takes as input his key-pair $\langle \mathsf{pk}, \mathsf{sk}\rangle$ and his choice v, and proceeds as follows:

1) The voter reads the list of eligible experts from the BB, denoted as $\mathsf{E} := \{\mathsf{E}_1, \ldots, \mathsf{E}_m\}$.
2) The voter computes $\boldsymbol{u} \leftarrow \mathsf{EC.Enc}_{\mathsf{pk}_\mathsf{T}}(v)$ and generates $\pi \leftarrow \mathsf{NIZK}_{\mathsf{knowledge}}.\mathsf{Prove}(\boldsymbol{u}, v)$, which is a proof of plaintext knowledge of $\boldsymbol{u}$.
3) The voter computes $\sigma \leftarrow \mathsf{Sig.sign}_{\mathsf{sk}}(\boldsymbol{u})$.
4) The voter sends the ballot $\beta_\mathsf{V} = (\mathsf{pk}, \boldsymbol{u}, \pi, \sigma)$ to the BB.

Procedure 6 ($\mathsf{E\_Vote}(\mathsf{Auth}, v)$). An expert casts his vote by encrypting his choice and authenticating to the BB.

1) The expert i computes $\boldsymbol{c} \leftarrow \mathsf{EC.Enc}_{\mathsf{pk}_\mathsf{T}}(v)$ and generates $\tau \leftarrow \mathsf{NIZK}_{\mathsf{knowledge}}.\mathsf{Prove}(\boldsymbol{c}, v)$, which is a proof of plaintext knowledge of $\boldsymbol{c}$.

2) The expert i uses Auth to generate an authencation message e_i on $\boldsymbol{c}$.
3) The expert i sends the ballot $\beta_{\mathsf{E}} = (\mathsf{E}_i, \boldsymbol{c}, \tau, e_i)$ to the BB.

Procedure 7 (Validate(BB, β)). The validation algorithm verifies that a ballot is valid with respect to the current state of the bulletin board.

1) If β is a voter's ballot, parse β as $(\mathsf{pk}, \boldsymbol{u}, \pi, \sigma)$. Check (i) pk does not appear in other ballots; (ii) $\boldsymbol{u}$ does not appear in other ballots; (iii) $\mathsf{NIZK}_{\mathsf{knowledge}}.\mathsf{Verify}(\pi, \boldsymbol{u}) \stackrel{?}{=} 1$; and (iv) $\mathsf{Sig.verify}_{\mathsf{pk}}(\sigma, \boldsymbol{u}) \stackrel{?}{=} 1$.
2) If β is an expert's ballot, parse β as $(\mathsf{E}_i, \boldsymbol{c}, \tau, e_i)$. Check (i) E_i does not appear in other ballots; (ii) $\boldsymbol{c}$ does not appear in other ballots; (iii) $\mathsf{NIZK}_{\mathsf{knowledge}}.\mathsf{Verify}(\tau, \boldsymbol{c}) \stackrel{?}{=} 1$; and (iv) e_i is E_i's valid authentication message on $\boldsymbol{c}$.
3) Return $\top$ if and only if all the checks pass.

Tally Phase:

Procedure 8 (Tally). When the voting phase ends, the shuffler and trustees will perform the tally procedure.

1) The shuffler takes all the registration items from the BB.
2) For each registration item $\langle \tilde{K}, A, \mathsf{tx}, \delta, \sigma_{\mathsf{RA}} \rangle$ published by the RA, check (i) $\mathsf{NIZK}_{\mathsf{power}}.\mathsf{Verify}(A, \mathsf{tx}, \delta) \stackrel{?}{=} 1$; (ii) $\mathsf{Sig.Verify}_{\mathsf{pk}_{\mathsf{RA}}}(\sigma_{\mathsf{RA}}, \tilde{K}||A||\mathsf{tx}||\delta) \stackrel{?}{=} 1$. The shuffler removes the invalid ones and removes $\mathsf{tx}, \delta, \sigma_{\mathsf{RA}}$.
3) For each fake registration item $\langle K', A_0, \rho \rangle$ published by a voter, check $\mathsf{NIZK}_{\mathsf{knowledge}}.\mathsf{Verify}(K', \rho) \stackrel{?}{=} 1$. Remove the invalid ones and remove ρ.
4) The shuffler takes all the ballots from the BB.
5) For each voter's ballot $\beta_{\mathsf{V}} = (\mathsf{pk}, \boldsymbol{u}, \pi, \sigma)$, check (i) $\mathsf{NIZK}_{\mathsf{knowledge}}.\mathsf{Verify}(\boldsymbol{u}, \pi) \stackrel{?}{=} 1$; (ii) $\mathsf{Sig.Verify}_{\mathsf{pk}}(\sigma, \boldsymbol{u}) \stackrel{?}{=} 1$. Remove the invalid ballots and remove π, σ. Now, a voter's ballot B is of the form $(\mathsf{pk}, \boldsymbol{u})$.
6) For each expert's ballot $\beta_{\mathsf{E}} = (\mathsf{E}_i, \boldsymbol{c}, \tau, e_i)$, check (i) $\mathsf{NIZK}_{\mathsf{knowledge}}.\mathsf{Verify}(\boldsymbol{c}, \tau) \stackrel{?}{=} 1$; (ii) e_i is a valid authentication message on $\boldsymbol{c}$. Remove the invalid ballots and remove τ, e_i. Now, an expert's ballot B_{E} is of the form $(\mathsf{E}_i, \boldsymbol{c})$.
7) The shuffler puts all the valid registration items together. At this point, each registration item is of the form (K, A), where K is the encrypted public key, and A is the encrypted voting power.
8) The shuffler verifiably shuffle re-encrypts the registration items.
9) For each shuffled registration item (K, A), the trustees verifiably decrypt the public keys K. If the same public key appears multiple times, drop all of them[2].
10) For each ballot B, if the public key does not match any public key in the registration items, drop it.
11) Put the corresponding A together with $\boldsymbol{u}$, i.e., assume a ballot $B = (\mathsf{pk}, \boldsymbol{u})$ and a registration item $W = (K, A)$, if $\mathsf{EC.Dec}_{\mathsf{sk}_{\mathsf{T}}}(K) = \mathsf{pk}$, we put A and $\boldsymbol{u}$ together to form a new item $I := (A, \boldsymbol{u})$.
12) The shuffler verifiably shuffle re-encrypts the new items.
13) For each candidate C_i, set initial score as $\boldsymbol{s}_i := \mathsf{LE.Enc}_{\mathsf{pk}_{\mathsf{T}}}(0)$. For each expert E_j, set initial score as $\boldsymbol{s}_{\ell+j} := \mathsf{LE.Enc}_{\mathsf{pk}_{\mathsf{T}}}(0)$.
14) For each shuffled item $I := (A, \boldsymbol{u})$, the trustees verifiably decrypt $\boldsymbol{u}$ to v and update candidate (or expert) v's score $\boldsymbol{s}_v := \boldsymbol{s}_v \cdot A$.
15) Form a list of each expert's encrypted score and encrypted candidate, i.e., expert E_i's entry is $(\boldsymbol{s}_{\ell+i}, \boldsymbol{c}_i)$.
16) For each expert's ballot $(\boldsymbol{s}, \boldsymbol{c})$, the trustees verifiably decrypt $\boldsymbol{c}$ to v and update candidate v's score $\boldsymbol{s}_v := \boldsymbol{s}_v \cdot \boldsymbol{s}$.
17) For each candidate C_i, the trustees verifiably decrypt the score $\boldsymbol{s}_i$ to s_i.
18) Return $\{s_i\}_{i \in [\ell]}$ as the final result.

Fig. 5 provides a graphical representation of the tally procedure (assuming there are n real ballots and n fake ballots).

Procedure 9 (VerifyElection). Any party can verify that the election is performed correctly.

1) Verify all experts' registration messages to check that the list of eligible experts is correctly formed.
2) For each registration item published by the RA, parse it as $\langle \tilde{K}, A, \mathsf{tx}, \delta, \sigma_{\mathsf{RA}} \rangle$, and check (i) $\mathsf{NIZK}_{\mathsf{power}}.\mathsf{Verify}(A, \mathsf{tx}, \delta) \stackrel{?}{=} 1$; (ii) $\mathsf{Sig.Verify}_{\mathsf{pk}_{\mathsf{RA}}}(\sigma_{\mathsf{RA}}, \tilde{K}||A||\mathsf{tx}||\delta) \stackrel{?}{=} 1$. Check that all the invalid ones are removed and all the valid ones are counted.
3) For each fake registration item published by a voter, parse it as $\langle K', A_0, \rho \rangle$, check $\mathsf{NIZK}_{\mathsf{knowledge}}.\mathsf{Verify}(K', \rho) \stackrel{?}{=} 1$. Check that all the invalid ones are removed and all the valid ones are counted.
4) Check all the shuffle NIZKs and decryption NIZKs during the tally phase.
5) Return $\top$ if and only if all of the checks pass.

How to deceive a coercer: Let $\langle K, A, \mathsf{tx}, \delta \rangle$ be the registration message, $\langle \tilde{K}, A, \mathsf{tx}, \delta, \sigma_{\mathsf{RA}} \rangle$ be the real registration item on BB, and pk' be the fake key used in the fake registration procedure. Once coerced, the voter casts a ballot using the fake key pk'. And he computes $K'' \leftarrow \mathsf{EC.Enc}_{\mathsf{pk}_{\mathsf{T}}}(\mathsf{pk}')$ and claims that $\tilde{K}$ is the re-encryption of K'' by simulating the designated verifier proof $\mathsf{NIZK}_{\mathsf{DVP\text{-}reenc}}$ (which requires the voter's blockchain secret key). The voter gives the simulated view to the coercer by replacing K with K'' and replacing the real proof with the simulated proof.

IV. Threat Model

We consider an adversary whose goal is to coerce voters into casting ballots for a particular candidate or to abstain. We first introduce the three underlying assumptions: 1) anonymous communication with honest BB, 2) untappable channels, and 3) inalienable authentication. While these assumptions are strong, we will argue that they are necessary assumptions for all "fake credentials" schemes and explain how they can be achieved in the blockchain context. Then, we give an informal

[2]This means that if a voter performs fake registration using his real key, his vote will be nullified.

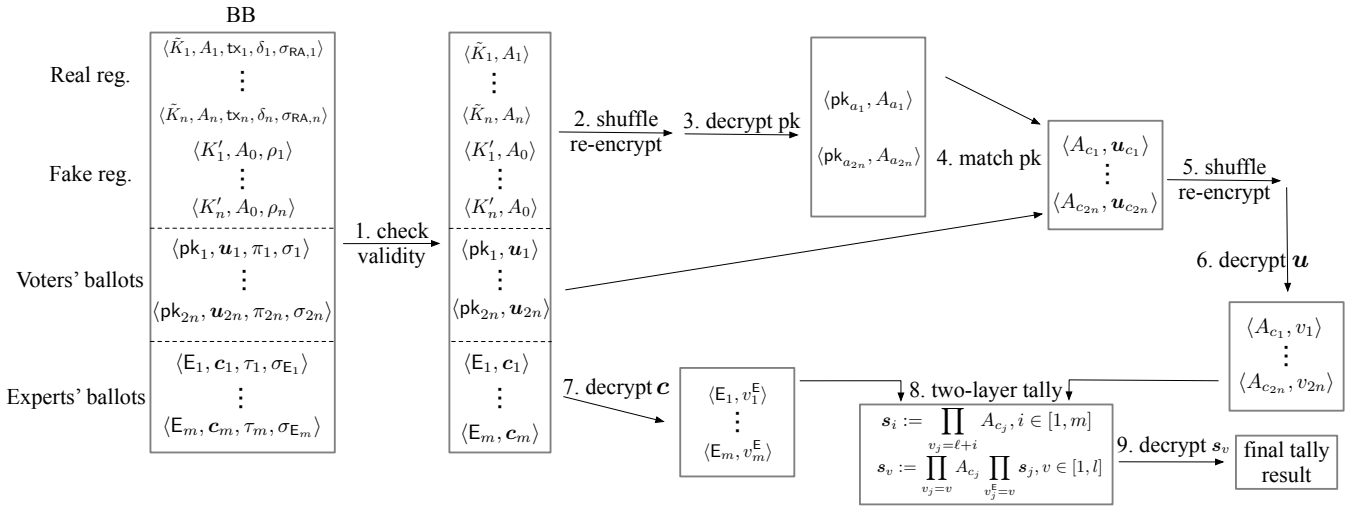

Fig. 5: Tally procedure (assuming there are n real ballots and n fake ballots)

analysis of how ballot privacy, verifiability, and coercion-resistance are achieved. We will formalize these properties and provide a security proof in section V.

A. Assumptions

Assumption 1. The bulletin board is honest, and the communication with the BB is anonymous.

BB is a basic assumption for any electronic voting scheme. A malicious BB can break verifiability by creating different views for different voters [31]. Then, ballot privacy will be undermined if ballots can be dropped undetectably [32], and it is not possible to achieve coercion-resistance without ballot privacy. Thus, BB is trusted for all three properties. Moreover, communication with BB must be anonymous; otherwise, the coercer will catch the deceiving voter when he tries to cast the real ballot.

In our system, the blockchain is a public ledger and serves as the honest BB. A voter can use anonymous channels (e.g., TOR) to broadcast on the blockchain. Although there are attacks that leverage the designs or implementation of the blockchain to deanonymize a user [33], [34], in this work, we assume that the communication with the blockchain can be anonymous, as in [35], [36]. How to achieve anonymous communication with the blockchain in practice is out of the scope of this paper.

Assumption 2. There is a secure (untappable) channel between the voter and the RA.

In all "fake credentials" schemes, a voter needs to establish a secret in the registration phase and keep the secret from the coercer. If the coercer taps all the communication between the voter and the authorities, then the voter's private information is a receipt/witness of what he cast [37].

As mentioned above, the RA can be instantiated with TEE, such as Intel SGX. A voter can communicate with the RA using TOR.

Assumption 3. The authentication is inalienable [16], i.e., the coercer cannot impersonate the voter or stop the voter from authenticating.

Inalienable authentication is a must for all voting schemes. Otherwise, the adversary can vote on the voter's behalf or launch a forced abstention attack.

In the blockchain context, the blockchain plays the role of PKI, and each voter authenticates to the RA by proving knowledge of his blockchain secret key. It is assumed that a voter will not reveal his blockchain secret key to the coercer, which will put the voter's stake at risk.

B. Security Properties

Definition 1. (Ballot privacy, informal) The adversary cannot learn the votes of honest voters.

Definition 2. (Verifiability, informal) Honest voters' ballots must be tallied, and the adversary cannot cast more votes than the number of voters that he controls.

Definition 3. (Coercion-resistance, informal) A coercer cannot determine if the coerced party is trying to deceive him.

In the following, we provide an intuitive explanation of why our scheme has ballot privacy, verifiability, and coercion-resistance.

Ballot privacy. In the registration phase, the voter himself generates the voting public key, and he encrypts it with the trustees' public key before sending it to the RA. Thus, nobody knows the link between the voting public key and the voter as long as the majority of trustees are honest. Voters' ballots are also encrypted with the trustees' public key. In the tally phase, ballots and registration items will be shuffled before decryption so that the link between identity and ballot/voting public key is broken by the shuffle. Therefore, ballot privacy is achieved if the shuffler and the majority of trustees are honest. Note: In delegated voting, usually the experts have input independence rather than ballot privacy, i.e., when casting the ballot, it should be independent of the others; later in the tally phase, it will be decrypted directly without shuffle. This is an essential requirement for delegated voting because we want to detect if an expert's behavior deviates from what he claimed.

Verifiability. A process composed of several subroutines is verifiable if each subroutine is verifiable itself. In the preparation phase, the trustees perform a verifiable distributed key

generation protocol [29]. In the registration phase, we use two NIZKs to ensure that (i) the encrypted voting power is equal to the frozen stake and (ii) the RA does the re-encryption correctly. In the fake registration process, $\mathsf{NIZK}_{\mathsf{sk}}$ makes sure that no one can nullify a public key without knowing the corresponding secret key. In the voting/delegation phase, the EUF-CMA property of the signature scheme prevents anyone who does not know the secret key from casting a valid ballot. In the tally phase, the shuffle NIZK [38] and the decryption NIZK [29] ensure that the final tally result is correct. In conclusion, all subroutines in our scheme are publicly verifiable, so no one needs to be trusted for verifiability.

Coercion-resistance. A coercer may ask the voter to reveal his real voting key pair, but the voter can claim a fake key pair as real by simulating the designated verifier proof, as long as the RA is not colluding with the coercer. In the tally phase, after shuffling all the registration items, real keys and fake keys become indistinguishable from the coercer's perspective.

Finally, Table III summarizes the trust assumptions on the entities for achieving each property.

TABLE III: Trust assumptions on the entities.

	Ballot privacy	Verifiability	Coercion-resistance
BB	Trusted	Trusted	Trusted
RA	Untrusted	Untrusted	Trusted
Shuffler	Trusted	Untrusted	Trusted
Trustees	t-out-of-k	Untrusted	t-out-of-k

Note on the use of TEE: One may ask, why not use TEE for the entire voting scheme since TEE is already used to instantiate the RA? The answer is that TEE is only trusted for coercion-resistance, not trusted for ballot privacy and verifiability. As we discuss below, simply distributing the RA does not work in a fake credential scheme.

Note on distributing the shuffler and RA: To distribute the shuffler, a mixnet [26] can be utilized, and we can perform a cryptographic sortition [27] on the blockchain to select the mixnet nodes. The assumption becomes that at least one of the mixnet nodes is honest instead of trusting a single shuffler.

However, simply distributing the RA does not lead to a weaker assumption because the coercer can ask the voter to provide the entire view of the registration phase. Even if only one of the RA parties is colluding with the coercer, the voter who does not know which RA party is colluding cannot simulate the registration view with negligible fail probability [7]. Concretely, if the voter fakes a message sent by an RA member, then he will have at least $1/n$ probability of being caught (in the case that the RA member is malicious), where n is the number of RA parties. Therefore, it is better not to distribute the RA for "fake credentials" schemes. We suggest using a TEE to instantiate the RA on the blockchain.

V. Security Analysis

In this section, we formally prove that our scheme has ballot privacy, verifiability, and coercion-resistance. We adopt the definitions of ballot privacy and coercion-resistance by Bernhard *et al.* [39], [40], and we adapt Cortier *et al.*'s verifiability definition [41] to delegative and weighted voting.

A. Ballot Privacy

Ballot privacy is based on the definition by Bernhard *et al.* [39]. The ballot privacy experiment tracks two bulletin boards: BB_0 for the "real" world and BB_1 for the "fake" world, in which only one bulletin board is available to the adversary $\mathcal{A}$. The outcome of the election is always computed on the real bulletin board BB_0. The adversary controls the registration authority RA and a subset of voters, and his goal is to distinguish whether the tally result is evaluated over the "real" or "fake" world.

Formally, the ballot privacy experiment is depicted in Fig. 6. The adversary $\mathcal{A}$ takes as input RA's secret key $\mathsf{sk}_{\mathsf{RA}}$ (since he controls the RA), the trustees' public key pk_{T}, and the candidate list $\mathcal{C}$, and he can query the following oracles:

- $\mathcal{O}\text{voteLR}(\mathsf{pk}, \mathsf{sk}, v_0, v_1)$, which allows the adversary $\mathcal{A}$ to let a voter with $(\mathsf{pk}, \mathsf{sk})$ cast a vote for v_0 on BB_0 and a vote for v_1 on BB_1.
- $\mathcal{O}\text{cast}(\beta)$, which allows the adversary $\mathcal{A}$ to cast a ballot β (constructed by $\mathcal{A}$) on both BB_0 and BB_1.
- $\mathcal{O}\text{board}()$, which allows the adversary $\mathcal{A}$ to see the content of a bulletin board depending on the bit b.
- $\mathcal{O}\text{tally}()$, which allows the adversary $\mathcal{A}$ to see the tally result and proof of tally correctness. The result r is always evaluated by tallying BB_0, and the proof is simulated when $b = 1$.

$\text{Exp}^{BP}_{\mathcal{A},\mathcal{V}}[b](\lambda, \mathcal{C})$:
$((\mathsf{pk}_{\mathsf{RA}}, \mathsf{sk}_{\mathsf{RA}}), (\mathsf{pk}_{\mathsf{T}}, \mathsf{sk}_{\mathsf{T}})) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{C})$;
$b' \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{sk}_{\mathsf{RA}}, \mathsf{pk}_{\mathsf{T}}, \mathcal{C})$;
output b'.
Oracles:
$\mathcal{O}\text{voteLR}(\mathsf{pk}, \mathsf{sk}, v_0, v_1)$:
let $\beta_0 = \mathsf{V\_vote}(\mathsf{pk}, \mathsf{sk}, v_0)$ and $\beta_1 = \mathsf{V\_vote}(\mathsf{pk}, \mathsf{sk}, v_1)$;
if $\mathsf{Validate}(\mathsf{BB}_b, \beta_b) = \perp$ then return $\perp$;
else $\mathsf{BB}_0 \leftarrow \mathsf{BB}_0 || \beta_0$ and $\mathsf{BB}_1 \leftarrow \mathsf{BB}_1 || \beta_1$.
$\mathcal{O}\text{cast}(\beta)$:
if $\mathsf{Validate}(\mathsf{BB}_b, \beta) = \perp$ then return $\perp$;
else $\mathsf{BB}_0 \leftarrow \mathsf{BB}_0 || \beta$ and $\mathsf{BB}_1 \leftarrow \mathsf{BB}_1 || \beta$.
$\mathcal{O}\text{board}()$:
return BB_b.
$\mathcal{O}\text{tally}()$:
$(r, \Pi_0) \leftarrow \mathsf{Tally}(\mathsf{BB}_0, \mathsf{sk}_{\mathsf{T}})$;
$\Pi_1 = \mathsf{SimTally}(\mathsf{BB}_1, r)$;
return (r, Π_b).

Fig. 6: Ballot privacy experiment $\text{Exp}^{BP}_{\mathcal{A},\mathcal{V}}[b](\lambda, \mathcal{C})$

Definition 1. *Ballot privacy. Let* $\mathcal{V} = \{\mathsf{Setup}, \mathsf{Reg}, \mathsf{FakeReg}, \mathsf{E\_reg}, \mathsf{V\_vote}, \mathsf{E\_vote}, \mathsf{Validate}, \mathsf{Tally}, \mathsf{VerifyElection}\}$ *be an election scheme for a candidate list* $\mathcal{C}$ *and security parameter* λ*. We say that* $\mathcal{V}$ *meets ballot privacy if there exists a PPT simulation algorithm* $\mathsf{SimTally}$ *such that for any PPT adversary* $\mathcal{A}$*:*

$$|\Pr[\text{Exp}^{BP}_{\mathcal{A},\mathcal{V}}[0](\lambda, \mathcal{C}) = 1] - \Pr[\text{Exp}^{BP}_{\mathcal{A},\mathcal{V}}[1](\lambda, \mathcal{C}) = 1]|$$

is a negligible function in the security parameter λ*.*

Theorem 1. *Assume that the ElGamal encryption scheme is IND-CPA secure, the underlying NIZKs* $\mathsf{NIZK}_i, i \in \{\mathsf{power}, \mathsf{knowledge}, \mathsf{DVP\text{-}reenc}, \mathsf{sk}, \mathsf{shuffle}, \mathsf{Dec}\}$ *are complete, sound, and zero-knowledge,* $\mathsf{NIZK}_{\mathsf{knowledge}}$ *is*

simulation-sound extractable, and the signature scheme is EUF-CMA secure. The voting protocol described in section III provides ballot privacy.

Proof. To prove this theorem, we construct the $\mathsf{SimTally}$ algorithm and prove indistinguishability through a series of games.

SimTally algorithm. The $\mathsf{SimTally}(\mathsf{BB}, r)$ algorithm performs the Tally procedure on BB, except that 1) all the proofs are simulated and 2) in steps 14 and 17 of the Tally procedure, the decryption results are based on r.

Now, we prove the indistinguishability through a sequence of games. We start with the adversary interacting with the challenger with $b = 0$ and end up with the adversary interacting with the challenger with $b = 1$.

Game G_0**.** Let G_0 be the experiment $\mathrm{Exp}^{BP}_{\mathcal{A},\mathcal{V}}[0](\lambda, \mathcal{C})$ (see Fig. 6 and Definition 1).

Game G_1**.** Game G_1 is the same as G_0, except that all proofs in the Tally procedure are simulated.

Claim 1: Because of the zero-knowledge property of the proofs, G_1 and G_0 are indistinguishable.

Game G_2**.** Game G_2 is the same as G_1, except that $\mathcal{O}$board returns BB_1 instead of BB_0.

Claim 2: If the ElGamal encryption scheme is IND-CPA secure and the NIZK $\mathsf{NIZK}_{\mathsf{knowledge}}$ is complete, simulation-sound extractable, and zero-knowledge, then G_2 and G_1 are indistinguishable.

Proof: Let n be the number of $\mathcal{O}$voteLR calls made by the adversary $\mathcal{A}$. We now build a series of games $H_0, \ldots, H_n$ and prove indistinguishability by a hybrid argument. Game H_i tracks a bulletin board BB. When the adversary calls $\mathcal{O}\mathsf{cast}(\beta)$, the challenger performs $\mathsf{BB} \leftarrow \mathsf{BB}||\beta$; for the first i calls to $\mathcal{O}$voteLR, the challenger performs $\mathsf{BB} \leftarrow \mathsf{BB}||\beta_1$; for the remaining calls, the challenger performs $\mathsf{BB} \leftarrow \mathsf{BB}||\beta_0$. Note that $H_0 = G_1$ and $H_n = G_2$.

$\mathrm{Exp}^{\mathsf{NM\text{-}CPA}}_{\mathcal{A}}(\lambda)$:
  $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Keygen}(1^\lambda)$;
  $m_0, m_1 \leftarrow \mathcal{A}(\mathsf{pk})$;
  $b \leftarrow_{\$} \{0, 1\}$;
  $c^* \leftarrow \mathsf{Enc}_{\mathsf{pk}}(m_b)$;
  $b' \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{dec}}(\mathsf{pk}, c^*)$;
$\mathcal{O}\mathsf{dec}(\boldsymbol{c})$:
  if $c^* \in \boldsymbol{c}$ then return $\perp$;
  for each i, $m_i \leftarrow \mathsf{Dec}(\mathsf{sk}, c_i)$;
  return $\boldsymbol{m} = \{m_i\}_i$.

Fig. 7: NM-CPA experiment $\mathrm{Exp}^{\mathsf{NM\text{-}CPA}}_{\mathcal{A}}(\lambda)$

As proved by Bernhard *et al.* [42], an IND-CPA secure encryption with simulation-sound extractable ZKP is NM-CPA (non-melleable) secure. The NM-CPA experiment is depicted in Fig. 7. The adversary may only call the oracle $\mathcal{O}$dec once. The adversary wins if $b' = b$ and his advantage is defined as $|\Pr[b' = b] - 1/2|$.

Now, we show that if an adversary $\mathcal{A}_i$ can distinguish H_i from H_{i-1}, we can construct adversary $\mathcal{B}_i$ that breaks the NM-CPA property of the encryption scheme. Adversary $\mathcal{B}_i$ receives the public key pk from its challenger. At the start of the game, $\mathcal{B}_i$ performs the Setup procedure and sets $\mathsf{pk}_\mathsf{T} = \mathsf{pk}$. It answers the jth $\mathcal{O}$voteLR query as follows:

- If $j < i$, it sets $\mathsf{BB} \leftarrow \mathsf{BB}||\beta_1$.
- If $j = i$, it sends v_0, v_1 to the NM-CPA challenger and receives a challenge ciphertext c^*. It uses c^* to obtain a ballot β^* and sets $\mathsf{BB} \leftarrow \mathsf{BB}||\beta^*$.
- If $j > i$, it sets $\mathsf{BB} \leftarrow \mathsf{BB}||\beta_0$.

When the adversary $\mathcal{A}_i$ calls $\mathcal{O}$tally, $\mathcal{B}_i$ proceeds as follows. Let $\Gamma := (\boldsymbol{u}_{i_1}, \pi_{i_1}), \ldots, c^* = (\boldsymbol{u}_j, \pi_j), \ldots, (\boldsymbol{u}_{i_k}, \pi_{i_k})$ be the vote ciphertexts and proofs in the valid ballots. $\mathcal{B}_i$ queries $\mathcal{O}$dec using $\Gamma \backslash c^*$ and obtains $(v_{i_1}, \ldots, v_{j-1}, v_{j+1}, \ldots, v_{i_k}) = \mathcal{O}\mathrm{dec}(\Gamma \backslash c^*)$. It sets the decryption result as $(v_{i_1}, \ldots, v_{j-1}, v_0, v_{j+1}, \ldots, v_{i_k})$ and continues the Tally procedure by simulating the proofs.

We can see that if $b = 0$ in $\mathcal{B}_i$'s NM-CPA game, then $\mathcal{B}_i$ perfectly simulates H_{i-1}; if $b = 1$ in $\mathcal{B}_i$'s NM-CPA game, then $\mathcal{B}_i$ perfectly simulates H_i. Thus, $\mathcal{B}_i$ breaks NM-CPA games if $\mathcal{A}_i$ distinguishes H_i from H_{i-1}. By a standard hybrid argument, $H_0 = G_1$ is indistinguishable from $H_n = G_2$.

Observe that the adversary's view in G_2 is identical to the view in the experiment $\mathrm{Exp}^{BP}_{\mathcal{A},\mathcal{V}}[1](\lambda, \mathcal{C})$. This concludes the proof. □

We also prove that our scheme has strong consistency and strong correctness by Bernhard *et al.* [39], which are necessary for a voting scheme to guarantee ballot privacy. Intuitively, strong consistency ensures that the result r is equal to the result function applied directly to the valid ballots. Strong correctness ensures that no adversary can generate a bulletin board BB such that $\mathsf{Validate}(\mathsf{BB}, \beta) = \perp$ for an honestly generated ballot β. The strong consistency experiment and strong correctness experiment are depicted in Fig. 8 and Fig. 9.

$\mathrm{Exp}^{\mathsf{s\text{-}cons}}_{\mathcal{A},\mathcal{V}}(\lambda, \mathcal{C})$:
  $((\mathsf{pk}_{\mathsf{RA}}, \mathsf{sk}_{\mathsf{RA}}), (\mathsf{pk}_\mathsf{T}, \mathsf{sk}_\mathsf{T})) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{C})$;
  $\mathsf{BB} \leftarrow \mathcal{A}(\mathsf{sk}_{\mathsf{RA}}, \mathsf{pk}_\mathsf{T})$;
  $(r, \Pi) \leftarrow \mathsf{Tally}(\mathsf{BB}, \mathsf{sk}_\mathsf{T})$;
  if $r \neq \rho(\mathsf{Extract}(\mathsf{sk}_\mathsf{T}, \beta_1), \ldots, \mathsf{Extract}(\mathsf{sk}_\mathsf{T}, \beta_n))$
  then return $\top$ else return $\perp$.

Fig. 8: Strong consistency experiment $\mathrm{Exp}^{\mathsf{s\text{-}cons}}_{\mathcal{A},\mathcal{V}}(\lambda, \mathcal{C})$

$\mathrm{Exp}^{\mathsf{s\text{-}corr}}_{\mathcal{A},\mathcal{V}}(\lambda, \mathcal{C})$:
  $((\mathsf{pk}_{\mathsf{RA}}, \mathsf{sk}_{\mathsf{RA}}), (\mathsf{pk}_\mathsf{T}, \mathsf{sk}_\mathsf{T})) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{C})$;
  $(id, v, \mathsf{BB}) \leftarrow \mathcal{A}(\mathsf{sk}_{\mathsf{RA}}, \mathsf{pk}_\mathsf{T})$;
  $\beta \leftarrow \mathsf{V\_vote}(\mathsf{pk}_{id}, \mathsf{sk}_{id}, v)$;
  if $\mathsf{Validate}(\mathsf{BB}, \beta) = \perp$ then return $\top$ else return $\perp$.

Fig. 9: Strong correctness experiment $\mathrm{Exp}^{\mathsf{s\text{-}corr}}_{\mathcal{A},\mathcal{V}}(\lambda, \mathcal{C})$

Definition 2. *Strong consistency. Let* $\mathcal{V} = \{\mathsf{Setup}, \mathsf{Reg}, \mathsf{FakeReg}, \mathsf{E\_reg}, \mathsf{V\_vote}, \mathsf{E\_vote}, \mathsf{Validate}, \mathsf{Tally}, \mathsf{VerifyElection}\}$ *be an election scheme for a candidate list* $\mathcal{C}$*, security parameter* λ*, and result function* ρ*. We say that* $\mathcal{V}$ *meets strong consistency if there exists functions* Extract *and* ValidInd *that satisfy the following conditions:*

1) *For* $((\mathsf{pk}_{\mathsf{RA}}, \mathsf{sk}_{\mathsf{RA}}), (\mathsf{pk}_\mathsf{T}, \mathsf{sk}_\mathsf{T})) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{C})$*, for all* $(\mathsf{pk}, \mathsf{sk})$ *output by* $\mathsf{Reg}(id)$*, and for any ballot* $\beta \leftarrow \mathsf{V\_vote}(\mathsf{pk}, \mathsf{sk}, v)$*, we have* $\mathsf{Extract}(\mathsf{sk}_\mathsf{T}, \beta) = v$.
2) *For any bulletin board and ballot generated by any PPT adversary* $\mathcal{A}$ *such that* $(\mathsf{BB}, \beta) \leftarrow \mathcal{A}$ *and* $\mathsf{Validate}(\mathsf{BB}, \beta) = \top$*, then* $\mathsf{ValidInd}(\beta) = \top$.

3) *For any PPT adversary $\mathcal{A}$, the advantage $\Pr[\text{Exp}^{\text{s-cons}}_{\mathcal{A},\mathcal{V}}(\lambda,\mathcal{C}) = \top]$ is a negligible function in the security parameter λ.*

Definition 3. *Strong correctness. Let $\mathcal{V} = \{\mathsf{Setup}, \mathsf{Reg}, \mathsf{FakeReg}, \mathsf{E\_reg}, \mathsf{V\_vote}, \mathsf{E\_vote}, \mathsf{Validate}, \mathsf{Tally}, \mathsf{VerifyElection}\}$ be an election scheme for a candidate list $\mathcal{C}$, and security parameter λ. We say that $\mathcal{V}$ meets strong correctness if for any PPT adversary $\mathcal{A}$:*

$$\Pr[\text{Exp}^{\text{s-corr}}_{\mathcal{A},\mathcal{V}}(\lambda,\mathcal{C}) = \top]$$

is a negligible function in the security parameter λ.

Theorem 2. *Under the same assumptions as theorem 1, the voting protocol described in section III provides strong consistency and strong correctness.*

Proof. We first define the functions Extract and ValidInd as follows:

1) $\mathsf{Extract}(\mathsf{sk}_\mathsf{T}, \beta)$ takes as input the extraction key sk_T and the ballot $\beta = (\mathsf{pk}, \boldsymbol{u}, \pi, \sigma)$. It checks the proof and signature in β. It returns $\perp$ if any of these checks fail; otherwise, it computes $v \leftarrow \mathsf{EC.Dec}_{\mathsf{sk}_\mathsf{T}}(\boldsymbol{u})$ and returns (pk, v).
2) $\mathsf{ValidInd}(\beta)$ checks the proof and signature in β. It returns $\top$ if and only if the checks pass.

The first condition of strong consistency is satisfied by the completeness of the zero-knowledge proofs, the correctness of signatures, and the correctness of ElGamal encryption scheme.

The second condition of strong consistency is satisfied because ValidInd executes a strict subset of the checks in Validate.

For the third condition, by the correctness of ElGamal decryption and the correctness of shuffle, the election results output by the Tally algorithm and the result function ρ are identical.

To prove strong correctness, we observe that an honestly generated ballot is not appended to the bulletin board if the corresponding pk already appears in another ballot. By the EUF-CMA property of the signature scheme, the adversary $\mathcal{A}$ has negligible probability of generating a valid signature under pk without knowing sk, so $\mathcal{A}$'s advantage in $\text{Exp}^{\text{s-corr}}_{\mathcal{A},\mathcal{V}}(\lambda,\mathcal{C})$ is negligible. □

B. Verifiability

We adapt Cortier *et al.*'s verifiability definition [41] to our delegative and weighted voting. In a nutshell, a voting scheme is verifiable if the election result reflects the votes of 1) all honest voters/experts who checked their votes, which appear on the bulletin board; 2) at most n_c voters/experts who are controlled by the adversary; and 3) a subset of the votes by honest voters/experts that did not check if their ballots were cast correctly. In our scheme, the registration authority is not trusted for verifiability, which is known as strong verifiability [41].

The verifiability experiment is formally depicted in Fig. 10. We use $\mathbb{U}$ to denote the set of public-private key pairs and Corrupted to denote the set of corrupted voters/experts. Let

$\text{Exp}^{ver}_{\mathcal{A},\mathcal{V}}(\lambda,\mathcal{C})$:
$((\mathsf{pk}_\mathsf{RA}, \mathsf{sk}_\mathsf{RA}), (\mathsf{pk}_\mathsf{T}, \mathsf{sk}_\mathsf{T})) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{C})$;
$(\mathsf{BB}, r, \Pi) \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{sk}_\mathsf{RA}, \mathsf{sk}_\mathsf{T})$;
if $\mathsf{VerifyElection}(\mathsf{BB}, (r, \Pi)) = \perp$ then return $\perp$;
if $r = \perp$ then return $\perp$;
$H = \{(\mathsf{pk}_i^h, v_i^h, \alpha_i^h, *)\}_{i=1}^{n_h}$ and $H' = \{(\mathsf{pk}_i^{h'}, v_i^{h'}, \alpha_i^{h'}, *)\}_{i=1}^{n_{h'}}$;
if $\exists\{\mathsf{pk}_i^c, v_i^c, \alpha_i^c\}_{i=1}^{n_c} \subseteq \mathsf{Corrupted}, \exists\{(\mathsf{pk}_i^{h'}, v_i^{h'}, \alpha_i^{h'}, *)\}_{i=1}^{t} \subseteq H'$ s.t.
$r = \rho(\{\mathsf{pk}_i^h, v_i^h, \alpha_i^h\}_{i=1}^{n_h} \cup \{\mathsf{pk}_i^c, v_i^c, \alpha_i^c\}_{i=1}^{n_c} \cup \{\mathsf{pk}_i^{h'}, v_i^{h'}, \alpha_i^{h'}\}_{i=1}^{t})$
then return $\perp$ else return $\top$.
Oracles:
$\mathcal{O}\text{corrupt}(id)$:
if $(id, *, *) \notin \mathbb{U}$ then return $\perp$;
$\mathsf{Corrupted} \leftarrow \mathsf{Corrupted} \cup \{(\mathsf{pk}_{id}, *, \alpha_{id})\}$;
return $(\mathsf{pk}_{id}, \mathsf{sk}_{id})$;
$\mathcal{O}\text{vote}(id, v)$:
if $(id, *, *) \notin \mathbb{U}$ or $(id, *) \in \mathsf{Corrupted}$ then return $\perp$;
if id is voter then
$\beta \leftarrow \mathsf{V\_vote}(\mathsf{pk}_{id}, \mathsf{sk}_{id}, v)$;
else
$\beta \leftarrow \mathsf{E\_vote}(\mathsf{Auth}_{id}, v)$;
end if
$H \cup H' \leftarrow \mathsf{Update}(H \cup H', (id, v, \alpha_{id}, \beta))$;
return β.

Fig. 10: Verifiability experiment $\text{Exp}^{ver}_{\mathcal{A},\mathcal{V}}(\lambda,\mathcal{C})$

$H = \{(\mathsf{pk}_i^h, v_i^h, \alpha_i^h, *)\}_{i=1}^{n_h}$ correspond to voters/experts that have checked that their ballots will be counted and $H' = \{(\mathsf{pk}_i^{h'}, v_i^{h'}, \alpha_i^{h'}, *)\}_{i=1}^{n_{h'}}$ correspond to voters/experts that have not checked their ballots (for experts, $\alpha_i = \emptyset$). We adapt the result function ρ to the delegative and weighted voting setting, i.e., $\rho : (\mathbb{V} \vee \mathbb{E})^\tau \rightarrow \boldsymbol{R}$ takes as inputs voters' and experts' choices, and outputs the election result. The adversary $\mathcal{A}$ takes as input RA's secret key sk_RA, the trustees' secret key sk_T, and the candidate list $\mathcal{C}$, and he can query the following oracles:

- $\mathcal{O}\text{corrupt}(id)$, which allows the adversary $\mathcal{A}$ to corrupt a voter/expert id.
- $\mathcal{O}\text{vote}(id, v)$, which allows the adversary $\mathcal{A}$ to let an honest voter/expert id cast a vote for v.

Note that $\mathcal{A}$ does not need the $\mathcal{O}$register oracle since he controls the registration authority and thus can register arbitrarily. The adversary wins if the result r verifies but violates any of the following conditions: 1) for each voter/expert that has checked his ballot, the ballot is counted; 2) at most n_c corrupted votes are counted; 3) a subset of votes by honest voters/experts that did not check their ballots are counted.

Definition 4. *Verifiability. Let $\mathcal{V} = \{\mathsf{Setup}, \mathsf{Reg}, \mathsf{FakeReg}, \mathsf{E\_reg}, \mathsf{V\_vote}, \mathsf{E\_vote}, \mathsf{Validate}, \mathsf{Tally}, \mathsf{VerifyElection}\}$ be an election scheme for a candidate list $\mathcal{C}$ and security parameter λ. We say that $\mathcal{V}$ meets verifiability if for any PPT adversary $\mathcal{A}$:*

$$\Pr[\text{Exp}^{ver}_{\mathcal{A},\mathcal{V}}(\lambda,\mathcal{C}) = \top]$$

is a negligible function in the security parameter λ.

Theorem 3. *Assume that the underlying NIZKs $\mathsf{NIZK}_i, i \in \{\mathsf{power}, \mathsf{knowledge}, \mathsf{DVP\text{-}reenc}, \mathsf{sk}, \mathsf{shuffle}, \mathsf{Dec}\}$ are complete, sound, and zero-knowledge, and the signature scheme is EUF-CMA secure. The voting protocol described in section III provides verifiability.*

Proof. Let the adversary $\mathcal{A}$ output a set of registration items, a set of votes, the result r, and the tally proof Π. Let $T =$

$\{\beta_1, \ldots, \beta_n\}$ be the valid ballots on the BB. By the homomorphism of ElGamal encryption scheme and the soundness of the tally proofs, we can conclude that r is the correct tally of $T = \{\beta_1, \ldots, \beta_n\}$ if $\mathsf{VerifyElection}(\mathsf{BB}, (r, \Pi))$ returns $\top$.

Now, we prove that for each ballot $\beta \in T$, it is either not counted or the re-randomized ballot of one of the following sets:

- $H = \{(\mathsf{pk}_i^h, v_i^h, \alpha_i^h, *)\}_{i=1}^{n_h}$, the votes of the honest voters who have checked their ballots.
- $H' = \{(\mathsf{pk}_i^{h'}, v_i^{h'}, \alpha_i^{h'}, *)\}_{i=1}^{n_{h'}}$, the votes of the honest voters who have not checked their ballots.
- $\mathsf{Corrupted} = \{\mathsf{pk}_i^c, v_i^c, \alpha_i^c\}_{i=1}^{n_c}$, the votes of the corrupted voters.

The soundness of $\mathsf{NIZK}_{\mathsf{power}}$ and $\mathsf{NIZK}_{\mathsf{DVP\text{-}reenc}}$ ensures that the registration items are formed correctly. The EUF-CMA property of the signature scheme ensures that the adversary cannot create a valid signature for an honest voter. Therefore, for each valid ballot $\beta \in T$, if it does not correspond to the above three sets, either it will be dropped or will match a fake registration item in the Tally procedure; in both cases, it will not be counted.

We now prove that the adversary cannot remove the votes of the honest voters who have checked their ballots. By the soundness of $\mathsf{NIZK}_{\mathsf{sk}}$, the adversary cannot create a valid "fake registration item" for pk if he does not know the corresponding sk. Thus, all the valid ballots will match the corresponding real registration item and be counted in the tally phase.

Finally, if a corrupted voter generates a ballot that encrypts an invalid candidate, it will be dropped in the tally phase. We can conclude that if the result r verifies, then it must correspond to the result of the tally function ρ computed on all the votes by honest voters who checked their ballots, at most n_c votes cast by corrupted voters, and a subset of votes cast by honest voters who did not check their ballots. □

C. Coercion-Resistance

The definition of coercion-resistance is inspired by Bernhard *et al.* [40]. Similar to the ballot privacy definition, the coercion-resistance experiment tracks two bulletin boards for the "real" world and the "fake" world, respectively. The adversary's goal is to distinguish whether he is in the "real" or "fake" world.

The coercion-resistance experiment is depicted in Fig. 11. The adversary $\mathcal{A}$ takes as input RA's public key $\mathsf{pk}_{\mathsf{RA}}$, the trustees' public key pk_{T}, and the candidate list $\mathcal{C}$ (since RA and trustees are trusted for coercion-resistance). The adversary has access to $\mathcal{O}$voteLR, $\mathcal{O}$cast, $\mathcal{O}$board, and $\mathcal{O}$tally as in the ballot privacy experiment, along with an additional oracle $\mathcal{O}\text{receipt}(id, v_0, v_1)$. $\mathcal{O}\text{receipt}(id, v_0, v_1)$ allows the adversary $\mathcal{A}$ to let a voter id submit to coercion by voting for v_0 on BB_0, and resist coercion by voting for v_1 on BB_1. It will return the ballot cast under coercion β_b and a receipt. If the voter submits to coercion, the receipt is the real view during the registration and voting phase; if the voter resists coercion, the receipt is simulated by the algorithm SimView.

Definition 5. *Coercion-resistance. Let* $\mathcal{V} = \{\mathsf{Setup}, \mathsf{Reg}, \mathsf{FakeReg}, \mathsf{E\_reg}, \mathsf{V\_vote}, \mathsf{E\_vote}, \mathsf{Validate}, \mathsf{Tally}, \mathsf{VerifyElection}\}$ *be an election scheme for a candidate list* $\mathcal{C}$ *and security parameter* λ*. We say that* $\mathcal{V}$ *meets coercion-resistance if there exist PPT simulation algorithms* SimView *and* SimTally *such that for any PPT adversary* $\mathcal{A}$*:*

$$|\Pr[\mathrm{Exp}^{CR}_{\mathcal{A},\mathcal{V}}[0](\lambda, \mathcal{C}) = 1] - \Pr[\mathrm{Exp}^{CR}_{\mathcal{A},\mathcal{V}}[1](\lambda, \mathcal{C}) = 1]|$$

is a negligible function in the security parameter λ*.*

$\mathrm{Exp}^{CR}_{\mathcal{A},\mathcal{V}}[b](\lambda, \mathcal{C})$:
$((\mathsf{pk}_{\mathsf{RA}}, \mathsf{sk}_{\mathsf{RA}}), (\mathsf{pk}_{\mathsf{T}}, \mathsf{sk}_{\mathsf{T}})) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{C})$;
$b' \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pk}_{\mathsf{RA}}, \mathsf{pk}_{\mathsf{T}}, \mathcal{C})$;
Output b'.
Oracles:
$\mathcal{O}\text{voteLR}(id, v_0, v_1)$:
$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Reg}(id)$;
let $\beta_0 = \mathsf{V\_vote}(\mathsf{pk}, \mathsf{sk}, v_0)$ and $\beta_1 = \mathsf{V\_vote}(\mathsf{pk}, \mathsf{sk}, v_1)$;
if $\mathsf{Validate}(\mathsf{BB}_b, \beta_b) = \perp$ then return $\perp$;
else $\mathsf{BB}_0 \leftarrow \mathsf{BB}_0 || \beta_0$ and $\mathsf{BB}_1 \leftarrow \mathsf{BB}_1 || \beta_1$.
$\mathcal{O}\text{receipt}(id, v_0, v_1)$: % submit versus resist
$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Reg}(id)$;
$(\mathsf{pk}', \mathsf{sk}') \leftarrow \mathsf{FakeReg}(id)$;
let $\beta_0 = \mathsf{V\_vote}(\mathsf{pk}, \mathsf{sk}, v_0)$ and $\beta_1 = \mathsf{V\_vote}(\mathsf{pk}', \mathsf{sk}', v_0)$;
if $\mathsf{Validate}(\mathsf{BB}_b, \beta_b) = \perp$ then return $\perp$;
else $\mathsf{BB}_0 \leftarrow \mathsf{BB}_0 || \beta_0$ and $\mathsf{BB}_1 \leftarrow \mathsf{BB}_1 || \beta_1$.
if $b = 0$ then
  $\mathsf{receipt} = \mathsf{View}\{\mathsf{Reg}(id), \mathsf{V\_vote}(\mathsf{pk}, \mathsf{sk}, v_0)\}$;
else
  $\mathsf{receipt} = \mathsf{SimView}(\mathsf{sk}_{id})$;
end if
let $\beta'_0 = \mathsf{V\_vote}(\mathsf{pk}', \mathsf{sk}', v_1)$ and $\beta'_1 = \mathsf{V\_vote}(\mathsf{pk}, \mathsf{sk}, v_1)$;
if $\mathsf{Validate}(\mathsf{BB}_b, \beta'_b) = \perp$ then return $\perp$;
else $\mathsf{BB}_0 \leftarrow \mathsf{BB}_0 || \beta'_0$ and $\mathsf{BB}_1 \leftarrow \mathsf{BB}_1 || \beta'_1$.
return $(\beta_b, \mathsf{receipt})$.
$\mathcal{O}\text{cast}(\beta)$:
if $\mathsf{Validate}(BB_b, \beta) = \perp$ then return $\perp$;
else $\mathsf{BB}_0 \leftarrow \mathsf{BB}_0 || \beta$ and $\mathsf{BB}_1 \leftarrow \mathsf{BB}_1 || \beta$.
$\mathcal{O}\text{board}()$:
return BB_b.
$\mathcal{O}\text{tally}()$:
$(r, \Pi_0) \leftarrow \mathsf{Tally}(\mathsf{BB}_0, \mathsf{sk}_{\mathsf{T}})$;
$\Pi_1 = \mathsf{SimTally}(\mathsf{BB}_1, r)$;
return (r, Π_b).

Fig. 11: Coercion-resistance experiment $\mathrm{Exp}^{CR}_{\mathcal{A},\mathcal{V}}[b](\lambda, \mathcal{C})$

Theorem 4. *Assume that the ElGamal encryption scheme is IND-CPA secure, the underlying NIZKs* $\mathsf{NIZK}_i, i \in \{\mathsf{power}, \mathsf{knowledge}, \mathsf{DVP\text{-}reenc}, \mathsf{sk}, \mathsf{shuffle}, \mathsf{Dec}\}$ *are complete, sound, and zero-knowledge,* $\mathsf{NIZK}_{\mathsf{knowledge}}, \mathsf{NIZK}_{\mathsf{sk}}$ *are simulation-sound extractable, and the signature scheme is EUF-CMA secure. The voting protocol described in section III provides coercion-resistance.*

Proof. To prove this theorem, we first construct SimTally and SimView algorithms and prove indistinguishability through a series of games.

SimTally algorithm. The $\mathsf{SimTally}(\mathsf{BB}, r)$ algorithm is the same as the one we define in the proof of theorem 1. It performs the Tally procedure on BB, except that 1) all the proofs are simulated and 2) in steps 14 and 17 of the Tally procedure, the decryption results are based on r.

SimView algorithm. The $\mathsf{SimView}(\mathsf{sk}_{id})$ performs as follows. Let $\langle K, A, \mathsf{tx}, \delta \rangle$ be the registration message, $\langle \tilde{K}, A, \mathsf{tx}, \delta, \sigma_{\mathsf{RA}} \rangle$ be the real registration item on BB, and pk' be the fake key used in the fake registration procedure. It computes $K'' \leftarrow$

$\mathsf{EC.Enc}_{\mathsf{pk_T}}(\mathsf{pk}')$ and claims that $\tilde{K}$ is the re-encryption of K'' by simulating the designated verifier proof $\mathsf{NIZK}_{\mathsf{DVP\text{-}reenc}}$, which requires sk_{id}. It returns the simulated view by replacing K with K'' and replacing the real proof with the simulated proof.

Now, we prove the indistinguishability through a sequence of games. We start with the adversary interacting with the challenger with $b = 0$ and end up with the adversary interacting with the challenger with $b = 1$.

Game G_0**.** Let G_0 be the experiment $\mathrm{Exp}^{CR}_{\mathcal{A},\mathcal{V}}[0](\lambda, \mathcal{C})$ (see Fig. 11 and Definition 5).

Game G_1**.** Game G_1 is the same as G_0, except that all proofs in the Tally procedure are simulated.

Claim 1: Because of the zero-knowledge property of the proofs, G_1 and G_0 are indistinguishable.

Game G_2**.** Game G_2 is the same as G_1, except that 1) $\mathcal{O}$receipt returns β_1 and the simulated view, and 2) $\mathcal{O}$board returns BB_1 instead of BB_0.

Claim 2: If the ElGamal encryption scheme is IND-CPA secure, the NIZKs $\mathsf{NIZK}_i, i \in \{\mathsf{knowledge}, \mathsf{DVP\text{-}reenc}, \mathsf{sk}\}$ are complete, sound, and zero-knowledge, and $\mathsf{NIZK}_{\mathsf{sk}}$, $\mathsf{NIZK}_{\mathsf{knowledge}}$ are simulation-sound extractable, then G_2 and G_1 are indistinguishable.

Proof: First, the adversary cannot distinguish the real view and the simulated view in the registration phase because of the zero-knowledge property of the designated verifier proof.

Second, by the simulation-sound extractability of $\mathsf{NIZK}_{\mathsf{sk}}$, the adversary cannot duplicate a "fake registration item" by re-randomizing an existing one. Thus, he cannot distinguish a fake ballot from a real ballot by the Tally procedure.

Now, except for seeing the simulated view instead of the real view, the only difference between G_2 and G_1 is that the adversary sees different encrypted ballots. Then, the rest of the proof is very similar to the proof of theorem 1. We define a sequence of games that replace the ballots on BB_0 with the ballots on BB_1 one by one. By a standard hybrid argument, the indistinguishability between G_2 and G_1 is reduced to the NM-CPA property of the underlying encryption scheme.

Finally, we observe that the adversary's view in G_2 is identical to the view in the experiment $\mathrm{Exp}^{CR}_{\mathcal{A},\mathcal{V}}[1](\lambda, \mathcal{C})$. This concludes the proof. □

VI. Discussion

Design decisions. Instead of letting the RA generate a fixed number of fake credentials in the registration phase (e.g., [10], [12]), we allow voters to arbitrarily generate and publish fake credentials at any time before the tally phase. This is the key design decision that makes our protocol achieve full coercion-resistance while remaining linear complexity. The observation is as follows. If each voter has d fake credentials, then the coercer can force the voter to cast $d + 1$ ballots, thus breaking coercion-resistance. Even if the number of fake credentials follows a special distribution (e.g., binomial distribution or normal distribution), this problem cannot be completely eliminated. In our scheme, the fake registration process does not involve the RA, which allows a voter to have any number of fake credentials to defend against the above attack.

In the protocol description, we do not allow a voter to revote with the same pk. However, other policies, such as keeping the most recent vote, are also feasible.

Vote-buying via stake-buying. In blockchain voting, typically, a voter's voting power is proportional to his stake. Since blockchain coins are publicly traded in open exchanges, one may argue that it is always possible to realize vote-buying via stake-buying. However, acquiring sufficient voting power through stake-buying is impractical. For an adversary to be successful, he needs to purchase a substantial amount of stake. Here's the catch: when buying from an exchange, there is often limited availability of stakes. Furthermore, rapidly purchasing large volumes of stake will inevitably drive up the price due to the basic principles of supply and demand. This surge in price could put the adversary's capital at risk. In contrast, vote-buying is a much simpler method and is detrimental to the decision-making process. Our coercion-resistant voting scheme effectively prevents vote-buying on the blockchain.

Stake renting/smart contract vote-buying. Another attack on blockchain voting is to use a smart contract to "rent" stakes. The smart contract collects stakes, uses the stakes for voting, and returns them with an extra payment after the election. We defend against this attack by prohibiting contract accounts from participating in the voting.

Inalienable authentication. Coercion-resistant voting requires inalienable authentication [16], i.e., the coercer can neither impersonate the voter nor prevent the voter from authenticating. In the blockchain context, authentication is realized by proving knowledge of his blockchain secret key, and it is assumed that the voter will not give his secret key to the coercer. However, the "Dark DAO" attack [43], [44] is still possible if the coercer can use TEEs. Specifically, the coercer can set up a TEE running a "cryptocurrency wallet" and create an encumbered key. The remote attestation of the TEE can prove that the wallet will not steal the bribee's stake. Then, a voter can transfer his stake to the account and use his secret key to manage it. At the end of the election, the TEE ensures a fair exchange for voting-buying. As pointed out in [43], this problem is inherent in any remote voting scheme where the secret key is generated by the voter. Kelkar *et al.* [45] proposed two schemes to defend against such attacks, using TEEs and ASICs (Application-Specific Integrated Circuit), respectively. But neither of them is suitable in the blockchain context. In this work, we assume that authentication is inalienable. How to defend against this type of attack is out of the scope of this paper. We leave it as an interesting open question.

Complexity. We analyze the complexity of our protocol in terms of the number of exponentiations. In the *preparation phase*, the RA takes $O(1)$ time to generate the signing key pair, and each trustee takes $O(k)$ time to perform the DKG protocol, where k is the number of trustees. In the *registration phase*, a voter generates an NIZK proof of voting power correctness and verifies a designated verifier proof, which has $O(1)$ complexity. In the *voting/delegation phase*, the ballot generation has $O(1)$ complexity. In the *tally phase*, the shuffler shuffles the ballots and the registration items, which has $O(n)$ complexity (counting public-key cryptographic operations only), where n is the number of voters. Then, the trustees decrypt the

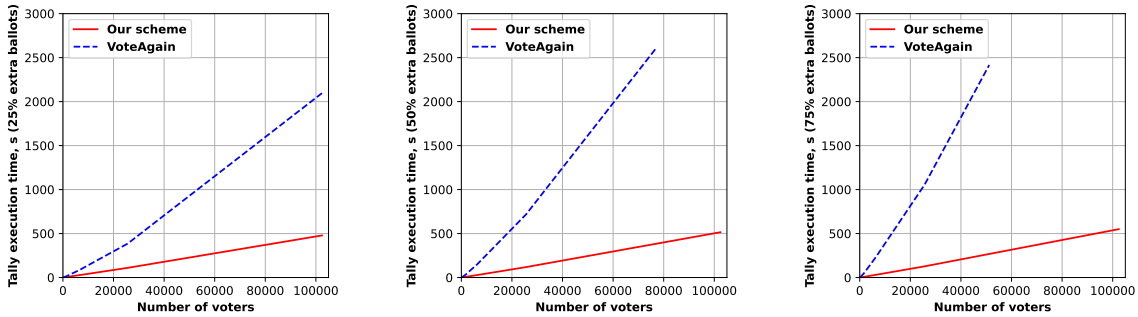

Fig. 12: Comparison of tally execution time between our scheme and VoteAgain [18] (with extra ballot rate as 25%, 50%, 75% from left to right). In VoteAgain, $x\%$ extra ballot rate means that $x\%$ voters re-vote once; in our scheme, $x\%$ extra ballot rate means that $x\%$ voters cast one fake ballot.

public keys, match the voting power with the candidates, and decrypt the candidates. Thus, the time complexity of a trustee is also $O(n)$. Adding them together, our scheme has $O(n)$ time complexity.

VII. Implementation and Evaluation

We implement a prototype of our voting scheme in Rust.[3] The implementation uses OpenSSL 1.1.1t to provide the basic elliptic curve math, and it uses Schnorr signature as the signature scheme. We evaluate all the cryptographic building blocks and the time consumption in each phase. The experiments are performed on a workstation with Intel Core i7-1165G7 @2.80GHz and 32GB RAM running Ubuntu 20.04.4 LTS x64, using the elliptic curve secp256r1.

Preparation phase. We evaluate the (t, k)-DKG execution time and traffic in the preparation phase, where t is the corruption threshold and k is the number of trustees. We set $t = k/2 - 1$, and the range of k is from 4 to 64. Results are given in Fig. 13.

Registration phase. In the registration phase, the voter uses $\mathsf{NIZK}_{\mathsf{power}}$ in the registration message to prove that the frozen stake is equal to the encrypted voting power, and the RA proves re-encryption correctness by a designated verifier proof $\mathsf{NIZK}_{\mathsf{DVP\text{-}reenc}}$. Generating a registration message costs 430.95 µs, and its size is 475 bytes. The designated verifier proof costs 102.91 µs to generate and 181.69 µs to verify, and its size is 141 bytes. Generating a fake registration item costs 336.05 µs, and the size is 267 bytes.

Voting/Delegation phase. In the voting/delegation phase, voters and experts encrypt the choice, sign it, and use $\mathsf{NIZK}_{\mathsf{knowledge}}$ to prove plaintext knowledge of the ballot. It takes 283.27 µs to generate a ballot. The size of a voter's ballot is 358 bytes, and the size of an expert's ballot is 332 bytes.

Tally phase. We evaluate the tally execution time with respect to different numbers of voters and different extra ballot rates and compare the results with VoteAgain [18]. Here, extra ballot rate represents how many voters cast extra ballots, i.e., in VoteAgain, 50% extra ballot rate means that 50% voters re-vote once; in our scheme, 50% extra ballot rate means that 50% voters cast one fake ballot. Note that, in our scheme, a voter's ballot takes more time to tally than an expert's ballot (because voters' ballots are shuffled and experts' ballots are not shuffled), so we set the number of experts as zero in the experiments, enabling comparable results with VoteAgain.

[3]The code is open source and can be found here: https://github.com/nastenko/coercion_resistant_voting_protocol.

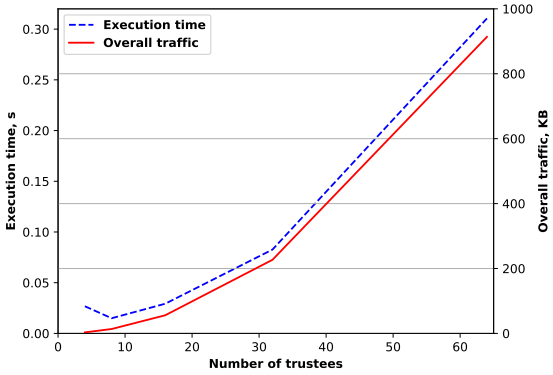

Fig. 13: (t, k)-DKG execution time and overall traffic with respect to different numbers of trustees, where $t = k/2 - 1$

Fig. 12 shows the tally execution time compared with VoteAgain [18] when the extra ballot rates are 25%, 50%, and 75% from left to right. VoteAgain's benchmark fails in 102400 voters with 50% and 75% extra ballot rates (probably because of too large ciphertext input). A higher rate of extra ballots confers a greater advantage to our scheme, as it necessitates VoteAgain to introduce a substantial quantity of dummy ballots in this scenario. In large-scale voting with more than 50,000 voters and over 50% extra ballot rate, our scheme's tally procedure is over 6x faster than VoteAgain.

VIII. Conclusion

In this work, we propose the first scalable coercion-resistant blockchain decision-making scheme that supports weighted votes and liquid democracy. It is scalable in the sense that it has constant ballot size and linear complexity. We formally prove that the proposed scheme has ballot privacy, verifiability, and coercion-resistance without any extra strong assumptions.

Our scheme has an advantage over all the existing coercion-resistant voting schemes, so it is suitable for large-scale coercion-resistant blockchain voting programs.

ACKNOWLEDGMENT

This work is supported by the National Natural Science Foundation of China (Grant No. 62232002) and by Input Output Global (IOG).

REFERENCES

[1] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, "Zerocash: Decentralized anonymous payments from bitcoin," in *2014 IEEE Symposium on Security and Privacy*. Berkeley, CA, USA: IEEE Computer Society Press, May 18–21, 2014, pp. 459–474.

[2] B. Ford, "Delegative democracy," https://bford.info/deleg/deleg.pdf, 2002, accessed: 2026-01-13.

[3] B. Zhang, R. Oliynykov, and H. Balogun, "A treasury system for cryptocurrencies: Enabling better collaborative intelligence," in *ISOC Network and Distributed System Security Symposium – NDSS 2019*. San Diego, CA, USA: The Internet Society, Feb. 24–27, 2019.

[4] Snapshot. (2024) Snapshot. [Online]. Available: https://snapshot.org

[5] B. Adida, "Helios: Web-based open-audit voting," in *USENIX Security 2008: 17th USENIX Security Symposium*, P. C. van Oorschot, Ed. San Jose, CA, USA: USENIX Association, Jul. 28 – Aug. 1, 2008, pp. 335–348. [Online]. Available: http://www.usenix.org/events/sec08/tech/full_papers/adida/adida.pdf

[6] P. Y. A. Ryan, D. Bismark, J. Heather, S. A. Schneider, and Z. Xia, "Prêt à voter: a voter-verifiable voting system," *IEEE Trans. Inf. Forensics Secur.*, vol. 4, no. 4, pp. 662–673, 2009. [Online]. Available: https://doi.org/10.1109/TIFS.2009.2033233

[7] A. Juels, D. Catalano, and M. Jakobsson, "Coercion-resistant electronic elections," in *Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society, WPES 2005, Alexandria, VA, USA, November 7, 2005*, V. Atluri, S. D. C. di Vimercati, and R. Dingledine, Eds. ACM, 2005, pp. 61–70. [Online]. Available: https://doi.org/10.1145/1102199.1102213

[8] R. Araújo, S. Foulle, and J. Traoré, "A practical and secure coercion-resistant scheme for remote elections," in *Frontiers of Electronic Voting, 29.07. - 03.08.2007*, ser. Dagstuhl Seminar Proceedings, D. Chaum, M. Kutylowski, R. L. Rivest, and P. Y. A. Ryan, Eds., vol. 07311. Internationales Begegnungs- und Forschungszentrum fuer Informatik (IBFI), Schloss Dagstuhl, Germany, 2007. [Online]. Available: http://drops.dagstuhl.de/opus/volltexte/2008/1295

[9] M. R. Clarkson, S. Chong, and A. C. Myers, "Civitas: Toward a secure voting system," in *2008 IEEE Symposium on Security and Privacy*. Oakland, CA, USA: IEEE Computer Society Press, May 18–21, 2008, pp. 354–368.

[10] R. E. Koenig, R. Haenni, and S. Fischli, "Preventing board flooding attacks in coercion-resistant electronic voting schemes," in *Future Challenges in Security and Privacy for Academia and Industry - 26th IFIP TC 11 International Information Security Conference, SEC 2011, Lucerne, Switzerland, June 7-9, 2011. Proceedings*, ser. IFIP Advances in Information and Communication Technology, J. Camenisch, S. Fischer-Hübner, Y. Murayama, A. Portmann, and C. Rieder, Eds., vol. 354. Springer, 2011, pp. 116–127. [Online]. Available: https://doi.org/10.1007/978-3-642-21424-0_10

[11] O. Spycher, R. E. Koenig, R. Haenni, and M. Schläpfer, "A new approach towards coercion-resistant remote e-voting in linear time," in *Financial Cryptography and Data Security - 15th International Conference, FC 2011, Gros Islet, St. Lucia, February 28 - March 4, 2011, Revised Selected Papers*, ser. Lecture Notes in Computer Science, G. Danezis, Ed., vol. 7035. Springer, 2011, pp. 182–189. [Online]. Available: https://doi.org/10.1007/978-3-642-27576-0_15

[12] J. Clark and U. Hengartner, "Selections: Internet voting with over-the-shoulder coercion-resistance," in *FC 2011: 15th International Conference on Financial Cryptography and Data Security*, ser. Lecture Notes in Computer Science, G. Danezis, Ed., vol. 7035. Gros Islet, St. Lucia: Springer Berlin Heidelberg, Germany, Feb. 28 – Mar. 4, 2012, pp. 47–61.

[13] R. Araújo and J. Traoré, "A practical coercion resistant voting scheme revisited," in *E-Voting and Identify - 4th International Conference, VoteID 2013, Guildford, UK, July 17-19, 2013. Proceedings*, ser. Lecture Notes in Computer Science, J. Heather, S. A. Schneider, and V. Teague, Eds., vol. 7985. Springer, 2013, pp. 193–209. [Online]. Available: https://doi.org/10.1007/978-3-642-39185-9_12

[14] R. Araújo, A. Barki, S. Brunet, and J. Traoré, "Remote electronic voting can be efficient, verifiable and coercion-resistant," in *FC 2016 Workshops*, ser. Lecture Notes in Computer Science, J. Clark, S. Meiklejohn, P. Y. A. Ryan, D. S. Wallach, M. Brenner, and K. Rohloff, Eds., vol. 9604. Christ Church, Barbados: Springer Berlin Heidelberg, Germany, Feb. 26, 2016, pp. 224–232.

[15] K. Gjøsteen, "The norwegian internet voting protocol," in *E-Voting and Identity - Third International Conference, VoteID 2011, Tallinn, Estonia, September 28-30, 2011, Revised Selected Papers*, ser. Lecture Notes in Computer Science, A. Kiayias and H. Lipmaa, Eds., vol. 7187. Springer, 2011, pp. 1–18. [Online]. Available: https://doi.org/10.1007/978-3-642-32747-6_1

[16] D. Achenbach, C. Kempka, B. Löwe, and J. Müller-Quade, "Improved Coercion-Resistant electronic elections through deniable Re-Voting," *USENIX Journal of Election Technology and Systems (JETS)*, Aug. 2015. [Online]. Available: https://www.usenix.org/conference/jets15/workshop-program/presentation/achenbach

[17] P. Locher, R. Haenni, and R. E. Koenig, "Coercion-resistant internet voting with everlasting privacy," in *FC 2016 Workshops*, ser. Lecture Notes in Computer Science, J. Clark, S. Meiklejohn, P. Y. A. Ryan, D. S. Wallach, M. Brenner, and K. Rohloff, Eds., vol. 9604. Christ Church, Barbados: Springer Berlin Heidelberg, Germany, Feb. 26, 2016, pp. 161–175.

[18] W. Lueks, I. Querejeta-Azurmendi, and C. Troncoso, "VoteAgain: A scalable coercion-resistant voting system," in *USENIX Security 2020: 29th USENIX Security Symposium*, S. Capkun and F. Roesner, Eds. USENIX Association, Aug. 12–14, 2020, pp. 1553–1570. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/lueks

[19] E. Magkos, M. Burmester, and V. Chrissikopoulos, "Receipt-freeness in large-scale elections without untappable channels," in *Towards The E-Society: E-Commerce, E-Business, and E-Government, The First IFIP Conference on E-Commerce, E-Business, E-Government (I3E 2001), October 3-5, Zürich, Switzerland*, ser. IFIP Conference Proceedings, B. F. Schmid, K. Stanoevska-Slabeva, and V. Tschammer, Eds., vol. 202. Kluwer, 2001, pp. 683–693. [Online]. Available: https://doi.org/10.1007/0-306-47009-8_50

[20] J. Alwen, R. Ostrovsky, H.-S. Zhou, and V. Zikas, "Incoercible multi-party computation and universally composable receipt-free voting," in *Advances in Cryptology – CRYPTO 2015, Part II*, ser. Lecture Notes in Computer Science, R. Gennaro and M. J. B. Robshaw, Eds., vol. 9216. Santa Barbara, CA, USA: Springer Berlin Heidelberg, Germany, Aug. 16–20, 2015, pp. 763–780.

[21] R. Giustolisi, M. S. Garjan, and C. Schürmann, "Thwarting last-minute voter coercion," in *2024 IEEE Symposium on Security and Privacy*. San Francisco, CA, USA: IEEE Computer Society Press, May 19–23, 2024, pp. 3423–3439.

[22] V. Cortier, P. Gaudry, and Q. Yang, "Is the JCJ voting system really coercion-resistant?" in *CSF 2024: IEEE 37th Computer Security Foundations Symposium*. Enschede, The Netherlands: IEEE Computer Society Press, Jul. 8–12, 2024, pp. 186–200.

[23] Votem. (2024) Votem. [Online]. Available: https://votem.com

[24] Y. Li, W. Susilo, G. Yang, Y. Yu, D. Liu, X. Du, and M. Guizani, "A blockchain-based self-tallying voting protocol in decentralized iot," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 1, pp. 119–130, 2022. [Online]. Available: https://doi.org/10.1109/TDSC.2020.2979856

[25] J. Benet, "IPFS - content addressed, versioned, P2P file system," *CoRR*, vol. abs/1407.3561, 2014. [Online]. Available: http://arxiv.org/abs/1407.3561

[26] D. Chaum, "Untraceable electronic mail, return addresses, and digital pseudonyms," *Commun. ACM*, vol. 24, no. 2, pp. 84–88, 1981. [Online]. Available: https://doi.org/10.1145/358549.358563

[27] J. Chen and S. Micali, "Algorand: A secure and efficient distributed ledger," *Theor. Comput. Sci.*, vol. 777, pp. 155–183, 2019. [Online]. Available: https://doi.org/10.1016/j.tcs.2019.02.001

[28] T. ElGamal, "A public key cryptosystem and a signature scheme based on discrete logarithms," *IEEE Transactions on Information Theory*, vol. 31, no. 4, pp. 469–472, 1985.

[29] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Secure distributed key generation for discrete-log based cryptosystems," in *Advances in Cryptology – EUROCRYPT'99*, ser. Lecture Notes in Computer Science,

J. Stern, Ed., vol. 1592. Prague, Czech Republic: Springer Berlin Heidelberg, Germany, May 2–6, 1999, pp. 295–310.
[30] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology – CRYPTO'86*, ser. Lecture Notes in Computer Science, A. M. Odlyzko, Ed., vol. 263. Santa Barbara, CA, USA: Springer Berlin Heidelberg, Germany, Aug. 1987, pp. 186–194.
[31] L. Hirschi, L. Schmid, and D. A. Basin, "Fixing the achilles heel of E-voting: The bulletin board," in *CSF 2021: IEEE 34th Computer Security Foundations Symposium*, R. Küsters and D. Naumann, Eds. Virtual Conference: IEEE Computer Society Press, Jun. 21–24, 2021, pp. 1–17.
[32] V. Cortier and J. Lallemand, "Voting: You can't have privacy without individual verifiability," in *ACM CCS 2018: 25th Conference on Computer and Communications Security*, D. Lie, M. Mannan, M. Backes, and X. Wang, Eds. Toronto, ON, Canada: ACM Press, Oct. 15–19, 2018, pp. 53–66.
[33] A. Biryukov and I. Pustogarov, "Bitcoin over Tor isn't a good idea," in *2015 IEEE Symposium on Security and Privacy*. San Jose, CA, USA: IEEE Computer Society Press, May 17–21, 2015, pp. 122–134.
[34] M. Kohlweiss, V. Madathil, K. Nayak, and A. Scafuro, "On the anonymity guarantees of anonymous proof-of-stake protocols," in *2021 IEEE Symposium on Security and Privacy*. San Francisco, CA, USA: IEEE Computer Society Press, May 24–27, 2021, pp. 1818–1833.
[35] T. Kerber, A. Kiayias, M. Kohlweiss, and V. Zikas, "Ouroboros crypsinous: Privacy-preserving proof-of-stake," in *2019 IEEE Symposium on Security and Privacy*. San Francisco, CA, USA: IEEE Computer Society Press, May 19–23, 2019, pp. 157–174.
[36] C. Ganesh, C. Orlandi, and D. Tschudi, "Proof-of-stake protocols for privacy-aware blockchains," in *Advances in Cryptology – EUROCRYPT 2019, Part I*, ser. Lecture Notes in Computer Science, Y. Ishai and V. Rijmen, Eds., vol. 11476. Darmstadt, Germany: Springer, Cham, Switzerland, May 19–23, 2019, pp. 690–719.
[37] M. Hirt and K. Sako, "Efficient receipt-free voting based on homomorphic encryption," in *Advances in Cryptology – EUROCRYPT 2000*, ser. Lecture Notes in Computer Science, B. Preneel, Ed., vol. 1807. Bruges, Belgium: Springer Berlin Heidelberg, Germany, May 14–18, 2000, pp. 539–556.
[38] S. Bayer and J. Groth, "Efficient zero-knowledge argument for correctness of a shuffle," in *Advances in Cryptology – EUROCRYPT 2012*, ser. Lecture Notes in Computer Science, D. Pointcheval and T. Johansson, Eds., vol. 7237. Cambridge, UK: Springer Berlin Heidelberg, Germany, Apr. 15–19, 2012, pp. 263–280.
[39] D. Bernhard, V. Cortier, D. Galindo, O. Pereira, and B. Warinschi, "SoK: A comprehensive analysis of game-based ballot privacy definitions," in *2015 IEEE Symposium on Security and Privacy*. San Jose, CA, USA: IEEE Computer Society Press, May 17–21, 2015, pp. 499–516.
[40] D. Bernhard, O. Kulyk, and M. Volkamer, "Security proofs for participation privacy, receipt-freeness and ballot privacy for the helios voting scheme," in *Proceedings of the 12th International Conference on Availability, Reliability and Security, Reggio Calabria, Italy, August 29 - September 01, 2017*. ACM, 2017, pp. 1:1–1:10. [Online]. Available: https://doi.org/10.1145/3098954.3098990
[41] V. Cortier, D. Galindo, S. Glondu, and M. Izabachène, "Election verifiability for helios under weaker trust assumptions," in *ESORICS 2014: 19th European Symposium on Research in Computer Security, Part II*, ser. Lecture Notes in Computer Science, M. Kutylowski and J. Vaidya, Eds., vol. 8713. Wroclaw, Poland: Springer, Cham, Switzerland, Sep. 7–11, 2014, pp. 327–344.
[42] D. Bernhard, O. Pereira, and B. Warinschi, "How not to prove yourself: Pitfalls of the Fiat-Shamir heuristic and applications to Helios," in *Advances in Cryptology – ASIACRYPT 2012*, ser. Lecture Notes in Computer Science, X. Wang and K. Sako, Eds., vol. 7658. Beijing, China: Springer Berlin Heidelberg, Germany, Dec. 2–6, 2012, pp. 626–643.
[43] P. Daian, T. Kell, I. Miers, and A. Juels. (2018) On-chain vote buying and the rise of dark daos. (Last accessed: 2024-04-18). [Online]. Available: https://hackingdistributed.com/2018/07/02/on-chain-vote-buying
[44] J. Austgen, A. Fábrega, S. Allen, K. Babel, M. Kelkar, and A. Juels, "DAO decentralization: Voting-bloc entropy, bribery, and dark daos," *CoRR*, vol. abs/2311.03530, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2311.03530
[45] M. Kelkar, K. Babel, P. Daian, J. Austgen, V. Buterin, and A. Juels, "Complete knowledge: Preventing encumbrance of cryptographic secrets," in *ACM CCS 2024: 31st Conference on Computer and Communications Security*, B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds. Salt Lake City, UT, USA: ACM Press, Oct. 14–18, 2024, pp. 2415–2429.
[46] M. Bellare and P. Rogaway, "Entity authentication and key distribution," in *Advances in Cryptology – CRYPTO'93*, ser. Lecture Notes in Computer Science, D. R. Stinson, Ed., vol. 773. Santa Barbara, CA, USA: Springer Berlin Heidelberg, Germany, Aug. 22–26, 1994, pp. 232–249.
[47] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016, Part II*, ser. Lecture Notes in Computer Science, M. Fischlin and J.-S. Coron, Eds., vol. 9666. Vienna, Austria: Springer Berlin Heidelberg, Germany, May 8–12, 2016, pp. 305–326.
[48] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *2018 IEEE Symposium on Security and Privacy*. San Francisco, CA, USA: IEEE Computer Society Press, May 21–23, 2018, pp. 315–334.
[49] C.-P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, Jan. 1991.
[50] R. Cramer, I. Damgård, and B. Schoenmakers, "Proofs of partial knowledge and simplified design of witness hiding protocols," in *Advances in Cryptology – CRYPTO'94*, ser. Lecture Notes in Computer Science, Y. Desmedt, Ed., vol. 839. Santa Barbara, CA, USA: Springer Berlin Heidelberg, Germany, Aug. 21–25, 1994, pp. 174–187.

APPENDIX A
DEFINITIONS

Here, we give formal security definitions of NIZKs, public-key encryption, and signatures. Our scheme deploys Sigma protocols with Fiat-Shamir heuristic to construct the NIZKs.

Sigma protocols. A Sigma protocol is a special type of 3-move public-coin proof system. A Sigma protocol for relation R consists of three algorithms $(\mathcal{C}, \mathcal{Z}, \mathcal{V})$:

- $a \leftarrow \mathcal{C}(x, w; r)$ takes as input the statement x, witness w and random coin r. It outputs the initial message a.
- $z \leftarrow \mathcal{Z}(x, w, r, e)$ takes as input the statement x, witness w, random coin r and a given challenge e. It outputs the answer z.
- $0/1 \leftarrow \mathcal{V}(x, a, e, z)$ takes as input the statement x, initial message a, challenge e and answer z. It outputs 0 or 1 for rejection or acceptance, respectively.

The completeness, k-special soundness, and special honest verifier zero-knowledge of Sigma protocols are defined as follows:

- Completeness: for all $(x, w) \in R, e \in \{0,1\}^\lambda$, $\Pr[a \leftarrow \mathcal{C}(x, w; r), z \leftarrow \mathcal{Z}(x, w, r, e) : \mathcal{V}(x, a, e, z) = 1] = 1$
- k-special soundness: there exists a deterministic polynomial-time extractor $\mathcal{X}$ such that for any PPT adversary $\mathcal{A}$: $\Pr[(x, a) \leftarrow \mathcal{A}(1^\lambda), e_1, \ldots, e_k \leftarrow \{0,1\}^\lambda, z_1, \ldots, z_k \leftarrow \mathcal{A}(e_1, \ldots, e_k), w \leftarrow \mathcal{X}(x, a, \{e_1, z_1\}, \ldots, \{e_k, z_k\}) : \mathcal{V}(x, a, e_i, z_i) = 1 \text{ for } i \in [k] \wedge R(x, w) = 0] \approx 0$.
- Special honest verifier zero-knowledge: there exists a PPT algorithm $\mathcal{S}$ such that for any PPT adversary $\mathcal{A}$: $\Pr[(x, e) \leftarrow \mathcal{A}(1^\lambda); a \leftarrow \mathcal{C}(x, w; r); z \leftarrow \mathcal{Z}(x, w, r, e) : \mathcal{A}(a, z) = 1] \approx \Pr[(x, e) \leftarrow \mathcal{A}(1^\lambda); (a, z) \leftarrow \mathcal{S}(x, e) : \mathcal{A}(a, z) = 1]$ where $(x, w) \in R$.

Fiat-Shamir Transformation [30]. Let $\Sigma = (\mathcal{C}, \mathcal{Z}, \mathcal{V})$ be a Sigma protocol and $\mathcal{H}$ a hash function. The Fiat-Sharmir transformation of Σ is the proof system $\mathsf{FS}_{\mathcal{H}}(\Sigma) = (\mathsf{Prove}, \mathsf{Verify})$ defined as follows:

- $(a, z) \leftarrow \mathsf{Prove}(x, w; r)$: Run $\mathcal{C}(x, w; r)$ to obtain initial message a. Compute $e \leftarrow \mathcal{H}(x, a)$. Run $\mathcal{Z}(x, w, r, e)$ to get the answer z. Output (a, z).
- $0/1 \leftarrow \mathsf{Verify}(x, a, z)$: Compute $e \leftarrow \mathcal{H}(x, a)$, then run $\mathcal{V}(x, a, e, z)$.

In the security proof, the hash function $\mathcal{H}$ is typically modeled as a random oracle. In the following, we give the definition of zero-knowledge [46] and simulation-sound extractability [42] (in the random oracle model).

Zero-knowledge [46]. A proof system $(\mathsf{Prove}, \mathsf{Verify})$ for relation $\mathcal{R}$ is zero-knowledge if there is a simulator $\mathcal{S}$ such that no adversary who can make queries to the random oracle and queries of the form $\mathsf{create\text{-}proof}(x, w)$ can distinguish the following two settings with non-negligibly better than 1/2 probability:

1) Random oracle queries are answered by a random oracle. In response to $\mathsf{create\text{-}proof}(x, w)$, the challenger checks that $\mathcal{R}(x, w)$. If not, he returns $\perp$. Otherwise, he returns $\mathsf{Prove}(x, w)$.
2) The challenger runs a copy of the simulator $\mathcal{S}$. It forwards random oracle queries to $\mathcal{S}$ directly. For $\mathsf{create\text{-}proof}(x, w)$, the challenger checks if $\mathcal{R}(x, w)$ holds: if not, the challenger returns $\perp$; if it holds, the challenger send $\mathsf{Simulate}(x)$ to $\mathcal{S}$ and returns the result to the adversary.

Simulation-sound extractability [42]. Let $\mathcal{P}$ be a zero-knowledge proof system with simulator $\mathcal{S}$. We say that $\mathcal{P}$ is simulation-sound extractable if there exists an extractor $\mathcal{K}$ such that for every prover $\mathcal{A}$, $\mathcal{K}$ wins the following game with overwhelming probability.

1) (Initial run.) The game selects a random string ω for $\mathcal{A}$. It runs an instance of $\mathcal{A}$ with the simulator $\mathcal{S}$ until $\mathcal{A}$ makes his output and halts. If $\mathcal{A}$ does not output any proofs, any of the proofs do not verify (w.r.t. the instance of $\mathcal{S}$ used as the random oracle) or any of $\mathcal{A}$'s statement/proof pairs (x, π) is such that π was the result of a $\mathsf{Simulate}(x)$ query, then $\mathcal{K}$ wins the game directly.
2) (Extraction.) The game runs an instance of $\mathcal{K}$, giving it the transcript of all queries in the initial run and the produced (x, π) as input. $\mathcal{K}$ may repeatedly make one type of query invoke in response to which the game runs a new invocation of $\mathcal{A}$ on the same randomness ω that it chose for the initial run. All queries made by these instances are forwarded to $\mathcal{K}$ who can reply to them.
3) $\mathcal{K}$ wins the game if it can output a vector of witnesses w that match the statements x of the initial run, i.e. for all i we have $\mathcal{R}(x_i, w_i)$.

Public-key encryption. A public-key encryption scheme consists of three PPT algorithms $(\mathsf{Keygen}, \mathsf{Enc}, \mathsf{Dec})$. The ElGamal encryption scheme we use is IND-CPA secure under the DDH assumption. Formally, consider the following IND-CPA experiment:

$\mathrm{Exp}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A},\mathsf{Enc}}(\lambda)$:

1) The challenger performs the key generation algorithm $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Keygen}(\lambda)$ and sends pk to the adversary $\mathcal{A}$.
2) $\mathcal{A}$ sends m_0, m_1 to the challenger.
3) The challenger picks a random bit $b \in \{0, 1\}$ and sends $c \leftarrow \mathsf{Enc}_{\mathsf{pk}}(m_b)$ to $\mathcal{A}$.
4) $\mathcal{A}$ outputs a guess bit $b' \in \{0, 1\}$. If $b = b'$, output 1; otherwise, output 0.

A public-key encryption scheme is IND-CPA secure if the adversary $\mathcal{A}$'s advantage $\mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{Enc}}(\mathcal{A}, \lambda) := |2 \cdot \Pr[\mathrm{Exp}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A},\mathsf{Enc}}(\lambda) = 1] - 1|$ is negligible in λ.

Signature. A signature scheme consists of three PPT algorithms $(\mathsf{Keygen}, \mathsf{Sign}, \mathsf{Verify})$. We require the underlying signature scheme to be existentially unforgeable under chosen message attack (EUF-CMA). The EUF-CMA experiment is as follows:

$\mathrm{Exp}^{\mathsf{EUF\text{-}CMA}}_{\mathcal{A},\mathsf{Sig}}(\lambda)$:

1) The challenger performs the key generation algorithm $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Keygen}(\lambda)$ and sends pk to the adversary $\mathcal{A}$.
2) $\mathcal{A}$ can repeatedly request for signatures on chosen messages $(m_0, \ldots, m_q)$, and receives the valid signatures $(\sigma_0, \ldots, \sigma_q)$ in response.
3) $\mathcal{A}$ outputs a message and signature (m^*, σ^*).
4) If m^* is not one of the messages requested in step 2, and $\mathsf{Verify}_{\mathsf{pk}}(m^*, \sigma^*) = 1$, output 1; otherwise, output 0.

A signature scheme is EUF-CMA if the adversary $\mathcal{A}$'s advantage $\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{Sig}}(\mathcal{A}, \lambda) := \Pr[\mathrm{Exp}^{\mathsf{EUF\text{-}CMA}}_{\mathcal{A},\mathsf{Sig}}(\lambda) = 1]$ is negligible in λ.

APPENDIX B
NIZKS

In this section, we show the construction of the NIZKs used in our system. There are six zero-knowledge proofs/arguments in our scheme for proving: (i) voting power correctness ($\mathsf{NIZK}_{\mathsf{power}}$); (ii) ElGamal encryption plaintext knowledge ($\mathsf{NIZK}_{\mathsf{knowledge}}$); (iii) re-encryption correctness ($\mathsf{NIZK}_{\mathsf{DVP\text{-}reenc}}$); (iv) knowledge of secret key ($\mathsf{NIZK}_{\mathsf{sk}}$) (v) shuffle correctness ($\mathsf{NIZK}_{\mathsf{shuffle}}$); and (vi) decryption correctness ($\mathsf{NIZK}_{\mathsf{Dec}}$). We adopt Bayer and Groth's scheme [38] for shuffle correctness and Gennaro *et al.*'s scheme [29] for decryption correctness. Here, we demonstrate the corresponding Sigma protocols. In our scheme, they are transformed into NIZKs by Fiat-Shamir heuristic [30]. By the result of Bernhard *et al.* [42], Fiat-Shamir transformation on Sigma protocols yields simulation-sound extractability in the random oracle model.

Proof of voting power correctness. In the registration phase, a voter will create a transaction tx that freezes his stake. Then, he sends the transaction tx and the encrypted voting power A to the RA and proves that the encrypted voting power is the same as the value of tx. In a privacy-preserving blockchain cryptocurrency system, tx usually contains an encrypted transaction value v. In this case, the zero-knowledge proof proves that A and v encrypt the same value. If the transaction value is encrypted by lifted Elgamal encryption scheme, then Fig. 14 shows the Sigma protocol for voting power correctness. If it is encrypted by a hybrid encryption scheme (e.g., ZCash [1]), we can utilize other zero-knowledge protocols for general circuits (e.g. Groth16 [47], Bulletproofs [48]).

Proof of ElGamal encryption plaintext knowledge. In the voting phase, we use a NIZK for ballot plaintext knowledge to prevent copying the other voter's choice. This can be proved with the Sigma protocol depicted in Fig. 15.

Designated verifier proof of re-encryption correctness. In the registration phase, the RA needs to generate a designated verifier proof for re-encryption correctness. To achieve the

designated verifier property, the statement is: (this is a correct re-encryption) OR (I know the verifier's blockchain secret key). The former part can be realized by the protocol depicted in Fig. 16, the latter part can be realized by the Schnorr protocol [49]. By the CDS composition [50], we can construct an OR-proof.

Proof of knowledge of secret key. To publish a fake registration item on the BB, the voter needs to prove knowledge of the corresponding secret key. This is a variant of the Schnorr protocol [49], depicted in Fig. 17.

Sigma protocol for voting power correctness

CRS: g, h, m.
Statement: $A = (A_1, A_2), v = (v_1, v_2)$.
Witness: α, r_1, r_2 such that $A = (g^{r_1}, g^{\alpha} h^{r_1}) \ \wedge \ v = (g^{r_2}, g^{\alpha} m^{r_2})$.

Prover:

- Pick random $\alpha', r_1', r_2' \leftarrow_{\$} \mathbb{Z}_p$;
- Compute $a_1 := g^{r_1'}, a_2 := g^{\alpha'} h^{r_1'}, a_3 := g^{r_2'}, a_4 := g^{\alpha'} m^{r_2'}$;
- $P \rightarrow V$: a_1, a_2, a_3, a_4.

Verifier:

- $V \rightarrow P$: random $e \leftarrow_{\$} \mathbb{Z}_p$.

Prover:

- Compute $z_1 := r_1' + e \cdot r_1, z_2 := r_2' + e \cdot r_2, z_3 := \alpha' + e \cdot \alpha$;
- $P \rightarrow V$: z_1, z_2, z_3.

Verifier:

- Output 1 if and only if the following holds:
  - $g^{z_1} = a_1 \cdot A_1^e$;
  - $g^{z_3} h^{z_1} = a_2 \cdot A_2^e$;
  - $g^{z_2} = a_3 \cdot v_1^e$;
  - $g^{z_3} h^{z_2} = a_4 \cdot v_2^e$.

Fig. 14: Sigma protocol for voting power correctness

Sigma protocol for plaintext knowledge

CRS: g, h.
Statement: $c = (c_1, c_2)$.
Witness: m, r such that $c_1 = g^r \ \wedge \ c_2 = m \cdot h^r$.

Prover:

- Pick random $r' \leftarrow_{\$} \mathbb{Z}_p, m' \leftarrow_{\$} \mathbb{G}$;
- Compute $a_1 := g^{r'}, a_2 := m' \cdot h^{r'}$;
- $P \rightarrow V$: a_1, a_2.

Verifier:

- $V \rightarrow P$: random $e \leftarrow_{\$} \mathbb{Z}_p$.

Prover:

- Compute $z_1 := r' + e \cdot r, z_2 := m' \cdot m^e$;
- $P \rightarrow V$: z_1, z_2.

Verifier:

- Output 1 if and only if the following holds:
  - $g^{z_1} = a_1 \cdot c_1^e$;
  - $z_2 \cdot h^{z_1} = a_2 \cdot c_2^e$.

Fig. 15: Sigma protocol for ElGamal encryption plaintext knowledge

Sigma protocol for re-encryption correctness

CRS: g, h.
Statement: $u = (u_1, u_2), v = (v_1, v_2)$.
Witness: r such that $v_1 = u_1 \cdot g^r \ \wedge \ v_2 = u_2 \cdot h^r$.

Prover:

- Pick random $r' \leftarrow_{\$} \mathbb{Z}_p$;
- Compute $a_1 = g^{r'}, a_2 := h^{r'}$;
- $P \rightarrow V$: a_1, a_2.

Verifier:

- $V \rightarrow P$: random $e \leftarrow_{\$} \mathbb{Z}_p$.

Prover:

- Compute $z := r' + e \cdot r$;
- $P \rightarrow V$: z.

Verifier:

- Output 1 if and only if the following holds:
  - $g^z = a_1 \cdot (v_1/u_1)^e$;
  - $h^z = a_2 \cdot (v_2/u_2)^e$.

Fig. 16: Sigma protocol for re-encryption correctness

Sigma protocol for knowledge of secret key

CRS: g, h.
Statement: $c = (c_1, c_2)$.
Witness: x, r such that $c_1 = g^r \ \wedge \ c_2 = g^x \cdot h^r$.

Prover:

- Pick random $r' \leftarrow_{\$} \mathbb{Z}_p, x' \leftarrow_{\$} \mathbb{G}$;
- Compute $a_1 := g^{r'}, a_2 := g^{x'} \cdot h^{r'}$;
- $P \rightarrow V$: a_1, a_2.

Verifier:

- $V \rightarrow P$: random $e \leftarrow_{\$} \mathbb{Z}_p$.

Prover:

- Compute $z_1 := r' + e \cdot r, z_2 := x' + e \cdot x$;
- $P \rightarrow V$: z_1, z_2.

Verifier:

- Output 1 if and only if the following holds:
  - $g^{z_1} = a_1 \cdot c_1^e$;
  - $g^{z_2} \cdot h^{z_1} = a_2 \cdot c_2^e$.

Fig. 17: Sigma protocol for knowledge of secret key